Authenticated Approximate Nearest Neighbor Search in Outsourced Vector Databases

Yuanhang Zhou, Xinyi Zhang, Cheng Xu, Jianliang Xu
Department of Computer Science, Hong Kong Baptist University, Hong Kong SAR
{csyhzhou, csxyzhang, chengxu, xujl}@comp.hkbu.edu.hk

arXiv:2611.16516v1 [cs.DB] 12 Nov 2026

***Abstract*—Approximate Nearest Neighbor (ANN) search has become a fundamental query primitive in modern vector databases for managing large-scale embedding data. To support growing vector collections, organizations increasingly outsource database management and query processing to third-party clouds. However, untrusted service providers may return suboptimal or fabricated results, posing a serious threat to ANN search integrity. Existing authenticated ANN approaches face substantial scalability challenges. They typically require transmitting high-dimensional vectors for distance verification and rely on topology-agnostic authentication structures, resulting in excessive communication and verification overhead. In this paper, we propose VIDA, a novel and efficient scheme for authenticated graph-based ANN search. VIDA replaces raw vector transmission with compact vector commitments and employs an aggregated Inner Product Argument to efficiently verify multiple distance computations. Furthermore, we introduce MeST, a topology-aware authenticated data structure that aligns authentication paths with graph traversal, significantly reducing structural proof overhead. Additionally, we design a quantization-aware authentication strategy that exploits distance bounds for pruning, reducing the number of exact distances requiring cryptographic authentication. Extensive experiments demonstrate that the proposed schemes substantially improve the efficiency of authenticated ANN search, achieving up to a 6.6× speedup in end-to-end latency and reducing data transfer time by up to 99.2% compared with state-of-the-art methods.**

I. Introduction

Driven by the rapid advancement of deep learning, massive volumes of unstructured data, such as text, images, and videos, are routinely transformed into high-dimensional vector embeddings to capture their underlying semantic relationships. To efficiently manage and query these immense datasets, specialized vector databases (*e.g.*, Milvus [1], Qdrant [2], and Weaviate [3]) have emerged and are now widely deployed. Serving as the core retrieval engine for these systems, ANN search has established itself as a fundamental primitive in the era of artificial intelligence. Leveraging this capability, vector databases empower a wide range of data-intensive applications, including retrieval-augmented generation [4], recommendation systems [5], and semantic search [6]. To support ANN search on a massive scale, modern vector databases rely on advanced indexing structures, such as hashing [7], inverted files [8], and proximity graphs [9], [10]. In this paper, we focus on proximity-graph-based (or simply graph-based) indexing methods, as they achieve the best trade-off between search efficiency and recall [11].

With the exponential growth in dataset sizes and the continuous increase in vector dimensionality, maintaining massive vector databases locally has become prohibitively expensive for many data owners. Consequently, there is a growing trend to outsource vector data management to third-party cloud platforms, leveraging their scalable storage and computational capabilities to support ANN queries [12]–[14]. Despite the benefits of outsourcing, this paradigm introduces critical trust concerns regarding the execution integrity of the search service. For instance, a rational but dishonest platform might terminate the query process early and return suboptimal results to reduce computational overhead, thereby compromising the accuracy of the results. While authenticated query processing over outsourced data has been extensively investigated for traditional relational databases [15]–[17], it remains largely under-explored for high-dimensional vector databases.

To ensure the integrity of ANN search in outsourced vector databases, a common approach is to employ traditional authenticated data structures (ADS), such as the Merkle Hash Tree (MHT), to build an authenticated graph index [18], [19]. Under this approach, upon executing an ANN query, the cloud service provider constructs and returns a verification object (VO) that contains the graph traversal path and all the raw high-dimensional vectors evaluated during the search. The client can then verify the integrity of the query results by locally replaying the search process and recomputing the distances. However, existing methods [18], [19] necessitate the transmission of large volumes of raw high-dimensional vectors in the VO for local distance recomputation. Furthermore, their MHTs are constructed without considering the underlying topology of the graph index, resulting in inefficient authentication paths. Consequently, these limitations lead to an excessively large VO size, incurring high bandwidth costs and resulting in prohibitive communication overhead.

To overcome this challenge, we propose VIDA, a novel and efficient scheme for authenticated graph-based ANN search that effectively addresses the critical bandwidth bottleneck. Unlike traditional baselines that require the transmission of raw high-dimensional vectors, VIDA integrates established vector commitments and Inner Product Argument (IPA) into a novel authentication scheme tailored to graph-based ANN search, enabling compact distance verification without transmitting raw vectors. Furthermore, to address the challenge of verifying a massive number of distance computations during graph traversal, we propose a proof aggregation mechanism that compresses the verification tasks for multiple visited

nodes into a single IPA proof, significantly reducing client-side computational overhead.

Beyond optimizing data-level transmission, we further propose MeST, a topology-aware ADS that minimizes the structural proof size. MeST is built using a novel distance-aware minimum spanning tree to align authentication paths with graph traversal, thereby reducing the size of Merkle proofs required for verification. Even after these optimizations, the residual verification cost still scales with the number of distance-evaluated candidates. We therefore extend VIDA to support lower-bound-based quantization-aware graph search, in which pruned candidates are justified by compact authenticated quantization payloads, while only the surviving candidates require exact-distance authentication via the aggregated IPA proof. Extensive experiments demonstrate that our methods outperform baseline methods, achieving a substantial speedup in end-to-end latency and reducing data transfer by orders of magnitude in high-recall scenarios.

Our main contributions are summarized as follows:

- We propose VIDA, a novel verifiable graph-based ANN search scheme that integrates vector commitments and IPA to authenticate distances without transmitting raw high-dimensional vectors.
- We design a proof aggregation mechanism that compresses multiple inner-product verifications into a single compact proof, greatly reducing client-side computational overhead.
- We introduce MeST, a topology-aware ADS built via a distance-aware minimum spanning tree, which aligns verification paths with graph traversal to minimize Merkle proof size.
- We adapt VIDA to quantization-aware graph search, reducing the number of exact distances that require cryptographic authentication.
- We provide a formal security analysis and conduct extensive experiments on both real-world and synthetic datasets. The results show that our methods achieve up to a 6.6× speedup in end-to-end latency and a 99.2% reduction in data transfer time compared with state-of-the-art methods.

The rest of the paper is organized as follows. Section II introduces our research problem, including the system model and security definitions. Section III reviews the preliminaries and presents a strawman solution. We detail the design of VIDA in Section IV and the MeST index structure in Section V. Section VI further extends VIDA to support quantization-aware authentication. Section VII summarizes the security guarantees, followed by the experimental evaluations in Section VIII. Finally, we discuss related work in Section IX and conclude the paper in Section X.

II. Problem Definition

Approximate Nearest Neighbor Search. Consider a database $\mathcal{D}$ consisting of a set of data objects, each represented as $o_i = (ID_i, \mathbf{x}_i)$, where ID_i is a unique identifier and $\mathbf{x}_i$ is the embedding vector of the original data payload (*e.g.*, string, graph, or time series). The distance between any two objects is evaluated by a metric function $\delta(\cdot, \cdot)$. Formally, a k-Nearest Neighbor (kNN) query $Q = (k, \mathbf{x}_q)$ requests the k nearest neighbors of the query vector $\mathbf{x}_q$. The exact query result set, denoted as $R_{exact} \subseteq \mathcal{D}$, satisfies $|R_{exact}| = k$ and $\delta(\mathbf{x}_q, \mathbf{x}_i) \leq \delta(\mathbf{x}_q, \mathbf{x}_{i'})$ for any $o_i \in R_{exact}$ and any $o_{i'} \in \mathcal{D} \setminus R_{exact}$. In practice, exact kNN search is computationally prohibitive in high dimensions due to the curse of dimensionality [9]. This paper therefore focuses on Approximate Nearest Neighbor (ANN) search. By relaxing exactness, ANN algorithms achieve substantially higher query efficiency. Accuracy is measured by the recall rate: $\mathsf{Recall} = \frac{|R \cap R_{exact}|}{k}$, where R is the approximate result set returned by the ANN search. The goal of ANN search is to maximize recall while minimizing query latency.

Fig. 1. System Overview

In this paper, we study authenticated ANN search over high-dimensional embedding vectors. We primarily focus on deterministic graph-based ANN algorithms under Euclidean distance, cosine similarity, and inner product, which together cover the majority of real-world ANN queries.

System Model. Our system, as illustrated in Figure 1, comprises three entities: the Data Owner (DO), the Service Provider (SP), and the Query Client.

- **DO:** The DO holds the original database $\mathcal{D}$ and builds an ANN search index $\mathcal{I}$ over $\mathcal{D}$ using a designated deterministic ANN algorithm. To support verifiable query execution, the DO constructs an Authenticated Data Structure (ADS) $\mathcal{A}$ on top of $\mathcal{I}$. The DO then outsources both $\mathcal{I}$ and $\mathcal{A}$ to the SP. To establish trust and ensure the integrity of the ADS, the DO signs the digest of $\mathcal{A}$ and publishes the signature to the client.
- **SP:** Equipped with abundant computational resources, the SP hosts $\mathcal{I}$ and $\mathcal{A}$ and provides query services on behalf of the DO. Upon receiving a query Q from the client, the SP executes the designated ANN algorithm over $\mathcal{I}$ to compute the result R and constructs a verification object (VO) based on $\mathcal{A}$ to attest to its integrity.
- **Query Client:** The client is an end-user who issues search queries to the SP. Upon receiving the result R and its corresponding VO, the client verifies the integrity and correctness of R by validating the VO.

Threat Model. The main security threat arises because the SP is an untrusted third party that may deviate arbitrarily from the protocol. For example, it may return incorrect results to save computation by skipping neighbors or tampering with the index or vector values. In contrast, we assume that both the DO and the query client are honest and follow the protocol. To counter this threat, the SP must return a VO along with the query result R to prove execution integrity. At the same time, because exact kNN search is prohibitively expensive in high dimensions, we do not require the SP to return the exact result.

TABLE I
FREQUENTLY USED NOTATIONS

Symbol	Description
$\mathcal{D}, \mathcal{I}, \mathcal{A}$	vector database, graph-based ANN index, and ADS
v_i, Adj_i	node and adjacency list
$o_i, ID_i, \mathbf{x}_i$	data object, identifier, and embedding vector
$\mathbf{x}_q, R$	query vector and result set
v_{ep}, $\delta(\cdot,\cdot)$	entry point of $\mathcal{I}$, and distance metric function
H_i, H_{root}, σ	node hash, root hash of $\mathcal{A}$, and DO's signature
$C_i, \|\mathbf{x}_i\|$	vector commitment and norm of $\mathbf{x}_i$
IP_i, ρ	inner product of $\mathbf{x}_i$ and $\mathbf{x}_q$, and IPA proof
g_i	graph ADS
$V_{\text{graph}}, V_{\text{merkle}}$	set of visited nodes and set of proved nodes in MeST
$V_{\text{exa}}, V_{\text{qua}}$	distance-evaluated and quantization-pruned node sets

Therefore, we formally define the security of authenticated ANN query processing via security games.

Definition 2.1 (Security)**.** We say an authenticated ANN search protocol is *secure* if, for any probabilistic polynomial-time (PPT) adversary Adv, the adversary's success probability in the following experiment is negligible:

- **Setup:** The adversary Adv chooses a database $\mathcal{D}$ and a query Q. A trusted party runs the index and ADS generation protocol on $\mathcal{D}$.
- **Ideal Game ($\mathbf{Game}_{\mathsf{Ideal}}$):** A trusted party faithfully executes the deterministic ANN algorithm to obtain the ideal result R_{Ideal}. The game outputs R_{Ideal}.
- **Real Game ($\mathbf{Game}_{\mathsf{Real}}$):** The adversary Adv generates a result R_{Real} together with a VO. The trusted party runs the verification algorithm $\texttt{Verify}(Q, R_{\mathsf{Real}}, \text{VO})$. If it outputs `reject`, the game outputs $\perp$; otherwise, the game outputs R_{Real}.

We say Adv succeeds if it can produce a query result that passes verification but differs from the ideal result R_{Ideal}. Formally:

$$\Pr[(\mathbf{Game}_{\mathsf{Real}} \neq \perp) \wedge (\mathbf{Game}_{\mathsf{Real}} \neq \mathbf{Game}_{\mathsf{Ideal}})] \leq \mathsf{negl}(\lambda)$$

where λ is the security parameter.

III. PRELIMINARIES

In this section, we introduce background on graph-based ANN algorithms and a strawman approach for authenticated ANN query processing. The frequently used notations throughout this paper are summarized in Table I.

A. Graph-based ANN Algorithms

Graph-based algorithms have been recognized as the most effective category for high-dimensional ANN search. Representative index structures, such as HNSW [9] and NSG [10], have demonstrated superior efficiency and accuracy. Consequently, they have been widely adopted in large-scale vector database systems [1]–[3]. In general, these algorithms operate in two primary phases: offline index construction and online query processing.

During the index construction phase, given a database $\mathcal{D}$, the construction algorithm builds a proximity graph $\mathcal{I}$. Each node v_i in the graph corresponds to a data object $o_i \in \mathcal{D}$, while the edges capture the neighborhood proximity between data points. The index is usually built incrementally by inserting points sequentially, and each new point is connected to a set of selected neighbors subject to an out-degree limit. During the query processing phase, given an ANN query $Q = (k, \mathbf{x}_q)$, the search algorithm initiates a greedy traversal over the graph $\mathcal{I}$, starting from a designated entry node v_{ep}. Throughout the traversal, a priority queue of candidate nodes is maintained, typically ordered by their distance to the query vector $\mathbf{x}_q$. The algorithm iteratively expands the search frontier by extracting the most promising candidates and evaluating their unvisited neighbors. After the traversal terminates according to the specific stopping condition, the top-k closest nodes encountered during the search are returned as the ANN query result R.

B. Strawman Solution

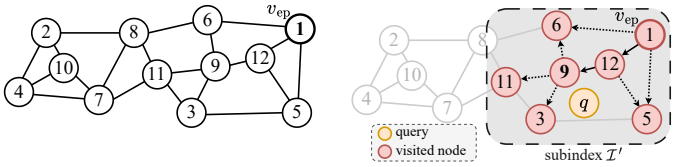
A strawman solution to enable authenticated graph-based ANN search is to build an MHT [20] as the underlying ADS over all graph nodes in $\mathcal{I}$. During the query phase, in addition to executing the prescribed ANN search algorithm, the SP is required to generate the corresponding Merkle proofs for all visited graph nodes. These visited nodes, along with their cryptographic proofs, constitute the verification object (VO). Upon receiving the VO, the client first verifies the data integrity of the visited nodes using their Merkle proofs. Subsequently, a subgraph is constructed from these verified nodes, allowing the client to locally replay the ANN search on this subgraph to verify the returned results.

1) ADS Construction: The ADS of the strawman solution is a binary MHT built on top of the graph nodes. Specifically, each graph node v_i is represented as a leaf node. The leaf node hash is computed as $H_i = \mathbb{H}(ID_i \mid Adj_i \mid \mathbf{x}_i)$, where ID_i and $\mathbf{x}_i$ are the identifier and raw vector of the corresponding data object, Adj_i is the adjacency list consisting of the identifiers of its neighbors on the index graph, $\mathbb{H}(\cdot)$ denotes a cryptographic hash function, and $\mid$ represents the concatenation operation. By contrast, the non-leaf node hash is computed as $\mathbb{H}(H_l \mid H_r)$, where H_l and H_r are the hashes of the left and right child nodes. Finally, the DO signs the root hash of the MHT and the identifier of the entry point (*i.e.*, $(H_{\text{root}}, ID_{\text{ep}})$) using its private key to generate a digital signature σ as the trusted root.

Example. *Consider a proximity graph as shown in Figure 2a. The strawman approach constructs a corresponding MHT, illustrated in Figure 3; e.g., the hash of leaf node v_1 is computed as $H_1 = \mathbb{H}(ID_1 \mid Adj_1 \mid \mathbf{x}_1)$, whereas the hash of the non-leaf node v_{13} is computed as $H_{13} = \mathbb{H}(H_1 \mid H_2)$. The DO signs $(H_{\text{root}}, ID_{\text{ep}} = 1)$ to generate σ.*

2) Verifiable Search: Given a query Q, the SP executes the designated graph-based ANN search and records the set of visited candidate nodes V_{graph} (*i.e.*, all nodes whose distances to the query vector are evaluated during the traversal, including the entry point v_{ep}). It then constructs a VO consisting of: (1) the identifier ID_i, raw vector $\mathbf{x}_i$, and graph adjacency list Adj_i for all $v_i \in V_{\text{graph}}$; (2) the corresponding Merkle proof for these nodes; and (3) the DO's signature σ. The SP returns the result set R along with the VO.

To verify the query result, the client first uses the Merkle proof in the VO to recompute the root hash from the received



(a) A Proximity Graph (b) An Example of Search

Fig. 2. An Example of Graph-based ANN search

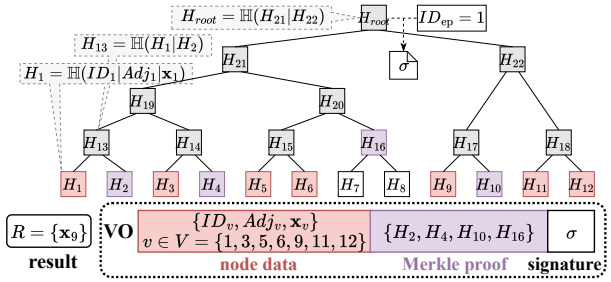


Fig. 3. Strawman Index

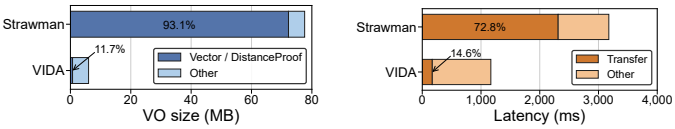
MHT nodes. It then authenticates the reconstructed hash against the one signed by the DO. This validates the integrity of all returned $v_i \in V_{\text{graph}}$. Next, the client assembles these authenticated nodes to build a local subgraph $\mathcal{I}'$. Finally, the client replays the search over $\mathcal{I}'$ to obtain a local result R'. The verification succeeds *iff* $R' = R$ and the local traversal does not try to access any unverified node outside V_{graph}.

Example. *In Figure 2b, for a 1-NN query Q, the traversal starts from entry point v_1 and visits the node set $V_{\text{graph}} = \{v_1, v_3, v_5, v_6, v_9, v_{11}, v_{12}\}$ to find the nearest neighbor v_9. Using the MHT in Figure 3, the SP generates a Merkle proof for all nodes in V_{graph}. To attest to the integrity of a single node (e.g., v_1), the standard Merkle proof requires its raw vector along with the sibling hashes $H_2, H_{14}, H_{20}, H_{22}$. When proving multiple nodes simultaneously, redundant hashes can be pruned by aggregating shared paths. For instance, nodes v_1 and v_3 share the sibling hashes H_{20}, H_{22} when recomputing $H_{19}, H_{21}, H_{\text{root}}$. The resulting minimal proof for V_{graph} is $\{H_2, H_4, H_{10}, H_{16}\}$, which must be sent to the client* ***alongside the raw high-dimensional vectors of all nodes in*** *V_{graph}.*

For result verification, the client first recomputes the leaf hashes H_1, H_3, H_5, H_6, H_9, H_{11}, H_{12} from the received node information. It then iteratively reconstructs the node hashes (e.g., H_{13}) by combining the recomputed leaf hashes with the provided sibling hashes until the Merkle root H_{root} is obtained. The client then checks σ against this reconstructed root, thereby verifying the correctness and integrity of the node data. Subsequently, the client assembles the authenticated nodes into a local subgraph (right part of Figure 2b). It then replays the search for Q over this subgraph from entry point v_1 and visits $v_5, v_6, v_{12}, v_9, v_3, v_{11}$ successively. The verification succeeds if the local result is $R' = \{\mathbf{x}_9\}$ and the traversal only accesses nodes within this subgraph.

3) Limitations: The strawman solution suffers from two main limitations that render it impractical for large-scale deployments.

Prohibitive Communication Overhead. The strawman approach generates an unacceptably large VO, imposing a severe burden on network bandwidth. To enable local search replay, the protocol must transmit the complete raw high-dimensional vector $\mathbf{x}_i$ for every visited node $v_i \in V_{\text{graph}}$. This bottleneck is especially severe in modern AI applications that use high-dimensional embeddings. To quantify the overhead, we conducted a preliminary evaluation on the OpenAI3072 dataset [21] ($d = 3072$). As illustrated in Figure 4, transmitting raw vectors constitutes the primary bottleneck of the strawman solution. Specifically, the raw vectors account for 93.1% of the 77.61 MB VO size. Consequently, data transfer alone consumes 72.8% of the end-to-end query latency.



(a) Breakdown of VO Size (b) Breakdown of Query Latency

Fig. 4. Preliminary Evaluation on OpenAI3072

Topology-Agnostic ADS. The MHT is constructed based on arbitrarily assigned node IDs and remains oblivious to the underlying proximity graph topology. Consequently, semantically close nodes that are frequently co-visited during search (*e.g.*, v_1, v_9, and v_{12} in Figure 2b) might be scattered across distant leaves of the MHT and share few common ancestors. This topological misalignment severely limits proof compression, leading to larger proof sizes and higher verification overhead. For instance, sequential nodes $\{v_1, v_2, v_3\}$ share deep common ancestors $H_{19}, H_{21}, H_{\text{root}}$, enabling compact aggregation, whereas the actual search trajectory $\{v_1, v_9, v_{12}\}$ shares only the root H_{root}. This structural deficiency results in substantially larger proofs and inflated verification costs.

IV. VIDA Scheme

In this section, we address the first challenge, namely the prohibitive communication overhead. To solve this issue, we propose VIDA, a Verifiable Inner-product-based Distance Authentication scheme that allows the client to authenticate distance computations without receiving the raw vectors.

A. Inner-product-based Distance Authentication

Preliminary evaluations in Figure 4 indicate that the primary bottleneck of the strawman solution is the requirement for the client to receive the complete raw vector $\mathbf{x}_i$ for every visited node $v_i \in V_{\text{graph}}$ to replay the search process locally. This overhead is particularly expensive for modern machine learning workloads, where the dimensionality of embeddings often reaches thousands and continues to grow. Ideally, it would be much preferable if the client could attest to the distance between $\mathbf{x}_q$ and $\mathbf{x}_i$ without receiving $\mathbf{x}_i$ itself. To achieve this, we analyze the algebraic structure of common distance metrics used in vector databases:

- *Euclidean distance:* $\left(\|\mathbf{x}_i\|^2 + \|\mathbf{x}_q\|^2 - 2\langle \mathbf{x}_i, \mathbf{x}_q \rangle\right)^{\frac{1}{2}}$.
- *Cosine similarity:* $\frac{\langle \mathbf{x}_i, \mathbf{x}_q \rangle}{\|\mathbf{x}_i\| \cdot \|\mathbf{x}_q\|}$.
- *Inner product:* $\langle \mathbf{x}_i, \mathbf{x}_q \rangle$.

A crucial observation is that these metrics depend on three components: the norm of the query vector $\|\mathbf{x}_q\|$, the norm of the data vector $\|\mathbf{x}_i\|$, and the inner product $IP_i = \langle \mathbf{x}_i, \mathbf{x}_q \rangle$.

Since the client provides the query vector $\mathbf{x}_q$, its norm $\|\mathbf{x}_q\|$ can be computed locally. Meanwhile, the DO can precompute the norm of each data vector $\|\mathbf{x}_i\|$ for every object $o_i \in \mathcal{D}$ as part of the ADS. Consequently, the remaining problem reduces to proving the correctness of the inner product IP_i. To this end, we propose to leverage the *Inner Product Argument (IPA)* [22], [23], a cryptographic protocol that allows a prover to convince a verifier of the inner product of two vectors represented by constant-size commitments. The IPA protocol consists of the following PPT algorithms:

- $\texttt{KeyGen}(1^\lambda) \to pp$: Given a security parameter 1^λ, it generates a public key pp.
- $\texttt{Com}(\mathbf{x}_i, pp) \to C_i$: Given a vector $\mathbf{x}_i$ and the public key pp, it generates a constant-size, computationally binding Pedersen vector commitment [24].
- $\texttt{ProveIPA}(\mathbf{x}_1, \mathbf{x}_2, pp) \to (IP, \rho)$: Given vectors $\mathbf{x}_1$, $\mathbf{x}_2$ and the public key pp, it computes the inner product $IP = \langle \mathbf{x}_1, \mathbf{x}_2 \rangle$ and the corresponding IPA proof ρ.
- $\texttt{VerifyIPA}(C_1, C_2, IP, \rho, pp) \to \{0, 1\}$: Given the commitments C_1, C_2, the claimed inner product IP, a proof ρ, and the public key pp, it outputs 1 if and only if $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = IP \wedge \texttt{Com}(\mathbf{x}_1) = C_1 \wedge \texttt{Com}(\mathbf{x}_2) = C_2$.

Because the size of a Pedersen commitment is independent of the vector dimensionality d, replacing raw vectors with commitments immediately removes the dominant communication overhead associated with transmitting $\mathbf{x}_i$. Furthermore, as we will see later, the IPA enables efficient verification of the inner product computations required for ANN queries and can be extended to support the aggregation of multiple proofs.

B. VIDA Design

We now describe how VIDA leverages inner-product-based distance authentication to support authenticated ANN search.

1) ADS Construction: The ADS in VIDA is structured similarly to the strawman solution, employing a binary MHT where each leaf node corresponds to a graph node v_i. However, the leaf node hash is computed as $H_i = \mathbb{H}(ID_i \,|\, Adj_i \,|\, C_i \,|\, \|\mathbf{x}_i\|)$, where C_i and $\|\mathbf{x}_i\|$ are the Pedersen commitment and the norm of $\mathbf{x}_i$, respectively. Note that including the raw vector $\mathbf{x}_i$ in the digest is unnecessary, as the commitment C_i itself serves as a cryptographic binding to the data object. The remainder of the ADS structure and its construction algorithm are identical to those described in Section III-B.

2) Verifiable Search: Given a query Q, the SP executes the designated graph-based ANN search over the index and records the visited node set V_{graph}. Unlike the strawman solution, for each visited node $v_i \in V_{\text{graph}}$, the VO includes the commitment C_i and norm $\|\mathbf{x}_i\|$ instead of the raw vector $\mathbf{x}_i$, while retaining the identifier ID_i and adjacency list Adj_i. Meanwhile, the SP generates an IPA proof ρ_i using $\texttt{ProveIPA}(\mathbf{x}_i, \mathbf{x}_q, pp)$ for each visited node v_i to attest to the inner product $IP_i = \langle \mathbf{x}_i, \mathbf{x}_q \rangle$. These authenticated inner products, combined with the commitments and norms, replace raw-vector transmission and enable the client to recompute distances without accessing $\mathbf{x}_i$, reducing communication overhead. Finally, the SP generates the Merkle proof for V_{graph} and returns the result set R along with the VO. In summary, the VO in VIDA comprises:

- the identifier ID_i, the adjacency list Adj_i, the commitment C_i, and the norm $\|\mathbf{x}_i\|$ for every $v_i \in V_{\text{graph}}$;
- the claimed inner product $IP_i = \langle \mathbf{x}_i, \mathbf{x}_q \rangle$ and corresponding IPA proof ρ_i for every $v_i \in V_{\text{graph}}$;
- the corresponding Merkle proof for V_{graph};
- the DO's signature σ.

Upon receiving (R, VO), the client first reconstructs the root hash and verifies the DO's signature σ. This step authenticates the search subgraph, including ID_i, Adj_i, C_i, and $\|\mathbf{x}_i\|$ for each visited node $v_i \in V_{\text{graph}}$. Subsequently, the client verifies each raw vector $\mathbf{x}_R$ in R against its corresponding commitment C_R. For each visited node $v_i \in V_{\text{graph}}$, the client validates the IPA proof ρ_i for the claimed inner product IP_i by invoking $\texttt{VerifyIPA}(C_i, C_q, IP_i, \rho_i, pp)$. A valid IPA proof certifies that $IP_i = \langle \mathbf{x}_i, \mathbf{x}_q \rangle$ matches the commitment C_i. Using the authenticated IP_i, the validated norm $\|\mathbf{x}_i\|$, and the locally computed $\|\mathbf{x}_q\|$, the client computes the distance $\delta(\mathbf{x}_i, \mathbf{x}_q)$ for each node v_i. It then constructs a local search subgraph $\mathcal{I}' = \{(ID_i, Adj_i, \delta_i)\}_{v_i \in V_{\text{graph}}}$ and replays the graph-based ANN search algorithm over $\mathcal{I}'$. Unlike the strawman solution, which recomputes distances from high-dimensional raw vectors, VIDA replays the search using these authenticated distances. Verification succeeds if the replay yields the same result set $R = R'$ without attempting to access any node outside the authenticated set V_{graph}.

Example. *The search in Figure 2b visits $V_{\text{graph}} = \{v_1, v_3, v_5, v_6, v_9, v_{11}, v_{12}\}$ and returns $R = \{\mathbf{x}_9\}$. The SP includes $(ID_i, Adj_i, C_i, \|\mathbf{x}_i\|)$ for every $v_i \in V_{\text{graph}}$ in the VO and provides the Merkle proof as in Section III-B2. For distance authentication, the SP computes $IP_1 = \langle \mathbf{x}_1, \mathbf{x}_q \rangle$ for v_1, generates the IPA proof ρ_1, and adds both to the VO instead of the raw vector $\mathbf{x}_1$. The client reconstructs the root from the node data and Merkle proof, verifies its signature, and checks each result vector in R against its commitment (e.g., $\texttt{Com}(\mathbf{x}_9) = C_9$). For every node in V_{graph}, it then verifies the IPA proof and recovers the distance. For v_1, it checks ρ_1 using C_1, IP_1, and the local query vector $\mathbf{x}_q$, then computes the Euclidean distance $\delta_1 = (\|\mathbf{x}_1\|^2 + \|\mathbf{x}_q\|^2 - 2IP_1)^{1/2}$.*

C. Proof Aggregation

Although the IPA-based approach significantly reduces the VO size, it requires a distinct proof for every visited node $v_i \in V_{\text{graph}}$. This imposes substantial computational overhead on the SP for proof generation and on the client for verification. Furthermore, high-recall ANN searches in machine learning workloads typically traverse thousands of nodes. The cumulative size and verification time of these individual proofs become a severe bottleneck. To address this, we propose an IPA aggregation protocol that compresses multiple proofs into a single proof. Specifically, it extends the original IPA with the following two PPT algorithms:

- $\texttt{ProveAggIPA}(\mathbf{x}_q, \{\mathbf{x}_i\}_{i=1}^n, pp) \to (\{IP_i\}_{i=1}^n, \rho_{agg})$: Given a common query vector $\mathbf{x}_q$, a set of n distinct vectors $\{\mathbf{x}_i\}_{i=1}^n$, and the public key pp, this algorithm computes

the n inner products $\{IP_i = \langle \mathbf{x}_q, \mathbf{x}_i \rangle\}_{i=1}^n$ and generates a single aggregated IPA proof ρ_{agg} that authenticates all of them simultaneously.

Our solution exploits the fact that all n inner products share the same query vector $\mathbf{x}_q$ as a common input. Thus, we apply random linear combination [25] to batch these n inner product arguments into one aggregated proof. Specifically, the prover first computes a random challenge r using the Fiat-Shamir heuristic [26] over the public transcript, which is derived from commitments and inner products corresponding to the query vector and n input vectors:

$$r = \mathbb{H}(C_q \mid C_1 \mid IP_1 \mid \cdots \mid C_n \mid IP_n). \tag{1}$$

The prover then computes an aggregated witness vector:

$$\mathbf{x}_{agg} = \sum_{i=1}^{n} r^i \cdot \mathbf{x}_i.$$

Finally, the prover invokes the standard IPA proving routine:

$$(IP_{agg}, \rho_{agg}) \leftarrow \texttt{ProveIPA}(\mathbf{x}_{agg}, \mathbf{x}_q, pp),$$

where ρ_{agg} serves as the aggregated IPA proof.

- $\texttt{VerifyAggIPA}(C_q, \{(C_i, IP_i)\}_{i=1}^n, \rho_{agg}, pp) \rightarrow \{0,1\}$: Given the commitment C_q corresponding to the query vector $\mathbf{x}_q$, a set of n pairs of commitments and inner products $\{(C_i, IP_i)\}_{i=1}^n$, the aggregated IPA proof ρ_{agg}, and the public key pp, this algorithm outputs 1 if and only if $\langle \mathbf{x}_q, \mathbf{x}_i \rangle = IP_i \wedge \texttt{Com}(\mathbf{x}_i) = C_i$ for all $i \in \{1, \ldots, n\}$.

The verification algorithm first recomputes the random challenge r using Equation (1). It then derives the aggregated commitment C_{agg} and the corresponding aggregated inner product IP_{agg}:

$$C_{agg} = \sum_{i=1}^{n} r^i \cdot C_i, \quad IP_{agg} = \sum_{i=1}^{n} r^i \cdot IP_i.$$

Finally, the verifier invokes the standard IPA verification routine:

$$\texttt{VerifyIPA}(C_q, C_{agg}, IP_{agg}, \rho_{agg}, pp).$$

The batch verification succeeds if and only if `VerifyIPA` returns 1.

We now describe the integration of the aggregation protocol into the verifiable search procedure of VIDA. Compared to the previous construction, where the SP generates an individual IPA proof for every visited node, we modify the distance-authentication proof generation step. Specifically, once the ANN search identifies the visited node set V_{graph}, the SP no longer invokes `ProveIPA` independently for each $v_i \in V_{\text{graph}}$. Instead, it aggregates the commitments $\{C_i\}_{v_i \in V_{\text{graph}}}$ and the corresponding inner products $\{IP_i\}_{v_i \in V_{\text{graph}}}$ and calls `ProveAggIPA` once, producing a single aggregated proof ρ_{agg}. Consequently, the multiple individual IPA proofs in the original VO are replaced by one compact proof, while the rest of the search algorithm remains unchanged.

During verification, instead of validating individual IPA proofs sequentially, the client invokes `VerifyAggIPA` to simultaneously authenticate all received inner products $\{IP_i\}_{v_i \in V_{\text{graph}}}$ using ρ_{agg}. If this batch verification succeeds, the client accepts the integrity of all inner-product claims and proceeds with the remainder of the verification procedure.

Example. *Continuing the previous example with $V_{\text{graph}} = \{v_1, v_3, v_5, v_6, v_9, v_{11}, v_{12}\}$, rather than generating seven individual proofs $\{\rho_1, \rho_3, \ldots, \rho_{12}\}$, the SP runs* `ProveAggIPA` *once to produce a single proof ρ_{agg} for the VO. During verification, the client computes $C_{agg} = rC_1 + r^2C_3 + \cdots + r^7C_{12}$ and $IP_{agg} = rIP_1 + r^2IP_3 + \cdots + r^7IP_{12}$, then uses ρ_{agg} to simultaneously validate all inner-product claims before continuing with the rest of the verification.*

V. Topology-aware VIDA

This section proposes a novel ADS called the Merkle Spanning Tree (MeST) for proximity graphs, which overcomes the topology-agnostic limitation mentioned in Section III-B3 and seamlessly integrates with VIDA.

A. Topology-aware ADS Construction

Recall that the strawman solution constructs the MHT based on node identifiers, ignoring the topological structure of the proximity graph. Consequently, it fails to exploit the search index's topological semantics, leading to redundant and inefficient Merkle proofs in the VO. To address this, we propose building the MHT directly over the proximity graph to align the ADS with the search access patterns. However, optimizing VO size for ANN search poses two primary challenges:

- *Graph-to-tree mapping and path compression:* Unlike the hierarchical structure of an MHT, proximity graphs are undirected and often cyclic. The challenge lies in finding an optimal graph-to-tree mapping such that nodes frequently co-visited during search share a common Merkle path with high probability. This alignment is critical for maximizing proof compression by deduplicating shared internal hash nodes.
- *Node content and minimal disclosure:* Since ANN search involves the iterative exploration of a node's neighborhood, the ADS must be carefully designed. We must determine the optimal granularity of information (*e.g.*, adjacency lists and commitments) stored in each MHT node to ensure the client receives all necessary data for verification while minimizing the number of revealed Merkle nodes to reduce VO size.

We observe that graph-based ANN search typically employs greedy traversal guided by a priority queue, where exploring a node frequently leads to the subsequent exploration of its immediate neighbors. To capture this spatial-topological correlation, we construct the MHT based on a spanning tree rooted at the graph's global entry point. While a standard unweighted minimum spanning tree can minimize hop distance by placing nodes reachable in d hops at depth d of the Merkle tree, it treats all neighbors equally and ignores the distance-driven nature of greedy search. Conversely, a traditional distance-weighted minimum spanning tree minimizes global edge weights without regard to tree depth, which may produce excessively deep paths and degrade Merkle proof efficiency. To balance these requirements, we propose a Distance-aware Minimum Spanning Tree (Algorithm 1). This approach treats minimizing hop distance as the primary

Algorithm 1: Distance-aware Minimum Spanning Tree

```
1  Function BuildSpanningTree(I):
2      Visited ← {v_ep}; Cur ← {v_ep};
3      while Cur ≠ ∅ do
4          Parent ← ∅;
5          foreach v_i ∈ Cur, v_j ∈ Adj_i do
6              if v_j ∉ Visited then
                   Parent[v_j] ← Parent[v_j] ∪ {v_i};
7          Next ← ∅;
8          foreach v_j ∈ Parent do
9              v_i* ← arg min_{v_i ∈ Parent[v_j]} δ(x_i, x_j);
10             Add edge (v_i*, v_j) to the spanning tree;
11             Visited ← Visited ∪ {v_j}; Next ← Next ∪ {v_j};
12         Cur ← Next;
```

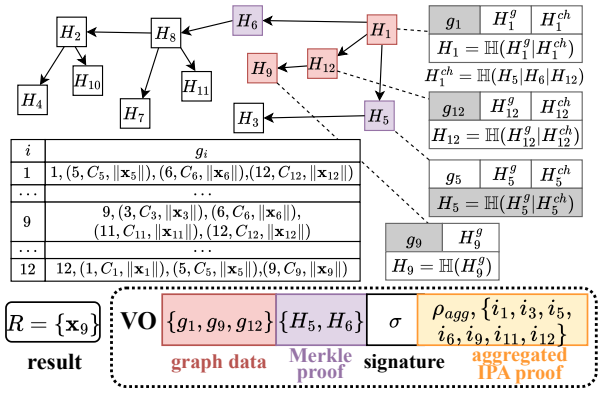


Fig. 5. MeST Index

global objective while employing a local greedy strategy to prioritize the closest neighbors as a tie-breaker when multiple choices exist (Line 9). By prioritizing local distance within a hop-constrained structure, the resulting ADS ensures that frequently co-visited nodes share common ancestor paths in the MHT, thereby maximizing proof compression in the VO.

Having defined the spanning tree structure, we now address the second challenge: determining the optimal data to store in each Merkle tree node to support verifiable search while revealing minimal information. Because the Merkle spanning tree allows graph nodes to reside in both internal and leaf positions, we first define a standardized Graph Node ADS:

Definition 5.1 (Graph Node ADS)**.** For each graph node v_i in the ANN search index $\mathcal{I}$, its corresponding ADS g_i consists of (i) ID_i, the identifier of the current node v_i; (ii) $\{(ID_j, C_j, \|\mathbf{x}_j\|)\}_{\forall v_j \in Adj_i}$, the identifiers, commitments, and norms of all immediate neighbors of v_i in $\mathcal{I}$. The hash of g_i is defined as $H_i^g = \mathbb{H}(ID_i \mid \{ID_j \mid C_j \mid \|\mathbf{x}_j\|\}_{\forall v_j \in Adj_i})$.

This design incorporates three key insights. First, the ADS g_i is based on the original proximity graph $\mathcal{I}$ rather than the Merkle spanning tree, so ANN search traverses the original index $\mathcal{I}$ while the Merkle tree only authenticates the underlying data. Second, embedding each neighbor's commitment C_j and norm $\|\mathbf{x}_j\|$ directly in g_i lets the client compute and authenticate $\delta(\mathbf{x}_q, \mathbf{x}_j)$ immediately upon receiving g_i. The full neighbor ADS g_j need not be revealed until that neighbor is visited, significantly reducing VO size. Finally, H_i^g uses a flattened representation of all neighbors. Providing this information together matches greedy ANN search, which examines all neighbors of an explored node to update the priority queue, and avoids the overhead of nested hash functions.

The construction of the remaining ADS is as follows. For each leaf node in the Merkle spanning tree, the hash is defined as $H_i = H_i^g$, directly using the corresponding graph node's ADS. In contrast, each internal node must encapsulate both its own graph node ADS and its children. Its hash is therefore defined as $H_i = \mathbb{H}(H_i^g \,|\, H_i^{ch})$, where H_i^{ch} denotes the hash of the children. Because the Merkle spanning tree often exhibits varying and occasionally high fanout at internal nodes, we compute H_i^{ch} using a threshold-based heuristic to optimize performance. If the fanout is below a predefined threshold, H_i^{ch} is computed as a flattened list hash: $H_i^{ch} = \mathbb{H}(H_{i,ch_1} \mid \cdots \mid H_{i,ch_m})$. Conversely, if the number of children is large, we construct a secondary binary MHT over these children and assign its root hash as H_i^{ch}. This hybrid approach ensures that when only a subset of children is visited during query processing, the verification remains efficient by minimizing the data required to reconstruct the Merkle root in the VO. Finally, to bind the entry point's own distance payload to the ADS, the DO computes $H_{\text{root}} = \mathbb{H}(H_{\text{ep}} \mid C_{\text{ep}} \mid \|\mathbf{x}_{\text{ep}}\|)$ and signs H_{root} as the trusted root of the entire ADS.

Example. *Following Figure 2a, we first construct the spanning tree via Algorithm 1 starting from v_1. The traversal first reaches nodes $\{v_5, v_6, v_{12}\}$ through v_1's adjacency list. It next discovers v_9, reachable from both v_6 and v_{12}, and selects v_{12} as its parent in the spanning tree by comparing $\delta(\mathbf{x}_9, \mathbf{x}_6)$ and $\delta(\mathbf{x}_9, \mathbf{x}_{12})$. By applying this rule to all nodes, we obtain the final spanning tree structure shown in Figure 5. On top of this tree, each node stores the ADS of the corresponding graph node. Concretely, for node v_1, its graph neighbors are v_5, v_6, and v_{12}. Hence, its graph ADS is $g_1 = \{1, (5, C_5, \|\mathbf{x}_5\|), (6, C_6, \|\mathbf{x}_6\|), (12, C_{12}, \|\mathbf{x}_{12}\|)\}$, and the corresponding graph digest is $H_1^g = \mathbb{H}(g_1)$.*

At the tree level, a leaf node directly uses its graph digest as its node hash. For example, v_9 is a leaf in the spanning tree, so $H_9 = H_9^g$. For a non-leaf node, MeST combines the graph-level digest with a digest of its children in the spanning tree. For example, the hash of node v_1 is computed as $H_1 = \mathbb{H}(H_1^g \mid H_1^{ch})$, where $H_1^{ch} = \mathbb{H}(H_5 \mid H_6 \mid H_{12})$ summarizes the hashes of the children of v_1 in the spanning tree. Applying this construction bottom-up to all nodes yields the root digest of the entire MeST index, namely $H_{\text{root}} = \mathbb{H}(H_1 \mid C_1 \mid \|\mathbf{x}_1\|)$, which is then signed by the DO to generate σ.

B. Verifiable Search

Next, we describe the verifiable search algorithm when integrated with MeST. As in Section IV-B2, the SP executes a priority-queue-guided graph traversal. However, in addition to recording V_{graph} for all visited graph nodes, the SP also tracks a separate set V_{merkle} that contains the nodes visited in the Merkle tree. As detailed in Algorithm 2, V_{graph} captures all nodes when they are pushed into the priority queue and have their distance to the query evaluated (Line 7) to ensure correct search results. In contrast, V_{merkle} only tracks the

Algorithm 2: MeST Verifiable Search

1 **Function** `Search`$(Q=(k,\mathbf{x}_q),\mathcal{I},\mathcal{A},\sigma)$**:**
2 $PriorityQueue \leftarrow \{(v_{\text{ep}},\delta_{\text{ep}})\}$; $V_{\text{merkle}} \leftarrow \{v_{\text{ep}}\}$; $V_{\text{graph}} \leftarrow \{v_{\text{ep}}\}$;
3 **while** $(v_i,\delta_i) \leftarrow PriorityQueue$.dequeue() **do**
4 Update result set R with v_i and stop if there are already k closer results;
5 $V_{\text{merkle}} \leftarrow V_{\text{merkle}} \cup \{v_i\}$;
6 **foreach** $v_j \in Adj_i$ **do**
7 $V_{\text{graph}} \leftarrow V_{\text{graph}} \cup \{v_j\}$;
8 $PriorityQueue$.enqueue$((v_j,\delta_j))$;
9 $\pi \leftarrow$ `ProveMerkle`$(V_{\text{merkle}},\mathcal{A})$;
10 $\rho_{agg} \leftarrow$ `ProveAggIPA`$(\mathbf{x}_q,\{\mathbf{x}_i\}_{v_i\in V_{\text{graph}}})$;
11 $VO \leftarrow (\{g_i\}_{v_i\in V_{\text{merkle}}},\pi,\sigma,\{IP_i\}_{v_i\in V_{\text{graph}}},\rho_{agg})$;
12 **return** (R,VO).

subset of nodes that are actually *explored* from the priority queue to examine their neighbors (Line 5). By maintaining this distinction, the SP can generate an aggregated IPA proof for all data vectors in the larger V_{graph}, while only requiring Merkle authentication paths for the significantly smaller V_{merkle}, thereby minimizing the overall VO size. The SP then outputs each graph ADS g_i for nodes in V_{merkle}, a Merkle proof for V_{merkle}, the original DO signature σ, all relevant inner products from V_{graph}, and the aggregated IPA proof ρ_{agg} as the VO.

On the client side, the verification procedure follows a logic similar to that in Section IV-B2. The primary distinction lies in the handling of the two node sets, V_{merkle} and V_{graph}. The client first reconstructs the root hash using the Merkle proof to authenticate the received graph ADS $\{g_i\}_{v_i\in V_{\text{merkle}}}$. Since each graph ADS g_i contains the commitments and norms for all its neighbors, the client can utilize them together with the inner products verified via ρ_{agg} to reconstruct the search subgraph for every node in V_{graph}. Finally, the client replays the ANN search over the locally reconstructed subgraph to verify the query result.

Example. *Consider the search process in Figure 2b. The search visits $V_{\text{graph}} = \{v_1, v_3, v_5, v_6, v_9, v_{11}, v_{12}\}$, while only the proven set $V_{\text{merkle}} = \{v_1, v_9, v_{12}\}$ needs to be covered by the Merkle proof in MeST. As shown by the graph ADS table in Figure 5, the remaining visited nodes $V_{\text{graph}} \setminus V_{\text{merkle}} = \{v_3, v_5, v_6, v_{11}\}$ are all exposed through the authenticated graph ADS of the proven nodes g_1, g_9, and g_{12}. Accordingly, the VO contains the graph ADS $\{g_1, g_9, g_{12}\}$, the required sibling hashes $\{H_5, H_6\}$, the signature σ, the claimed inner products $\{IP_i\}_{v_i\in V_{\text{graph}}}$, and the aggregated IPA proof ρ_{agg}.*

During verification, the client first reconstructs the Merkle root from $\{g_1, g_9, g_{12}\}$ and $\{H_5, H_6\}$, then verifies the signature σ. It next verifies ρ_{agg} and recovers the distances of all nodes in V_{graph}. Using the authenticated graph ADS of V_{merkle} together with the recovered distances for V_{graph}, the client constructs the local subgraph $\mathcal{I}'$ and replays the search.

C. MeST for Multi-layered Graphs

Many graph-based ANN indexes, such as HNSW [9], use multi-layered graphs. MeST supports them by building the Merkle spanning tree over the union of all layer-wise edges and organizing each node's graph ADS into layer-specific neighbor payloads whose digests are combined into the node hash. During verifiable search, the SP reveals only accessed-layer payloads and provides digests for the others. The client replays the original top-down search and rejects if any required payload is missing. Full details are in Appendix A.

VI. Quantization-aware VIDA

VIDA and MeST reduce authentication overhead through IPA aggregation and a topology-aware ADS, respectively. Nevertheless, each evaluated candidate introduces an exact-distance authentication instance for verifier-side replay, so the remaining cost scales with the candidate count. Large-scale and high-recall graph-based ANN searches may evaluate many candidates, shifting the dominant authentication cost from per-candidate overhead to the number of such instances.

Modern ANN systems address an analogous query-time bottleneck through vector quantization, which encodes vectors into compact representations and evaluates candidates without accessing their full-precision vectors. Classical methods like PQ [8] and SQ [27] approximate distances from compressed codes and often rerank a shortlist using exact distances to preserve search accuracy. In contrast, methods such as RaBitQ [28] compute one-sided lower bounds that enable pruning when they exceed the current search threshold. Such pruning is more compatible with authenticated search, because the verifier can recompute the bound and validate the pruning decision without checking the exact distance.

Motivated by this observation, we extend VIDA to authenticate quantization-aware graph search. Exact distances are authenticated only for candidates that require exact evaluation, whereas candidates eliminated by quantization can be justified using compact authenticated quantization payloads. Quantization thus reduces both ANN query-time computation and the number of exact distance computations that must enter the aggregated IPA proof. We next present the corresponding ADS construction and verifiable search protocol.

ADS Construction. To support quantization-aware authentication, we extend the Graph Node ADS defined in Section V-A. Besides the exact-distance payload used for IPA-based authentication, each neighbor entry must also bind the quantization payload used to compute the lower bound. Accordingly, each neighbor v_j contributes two payloads to the quantization-aware Graph Node ADS: the exact-distance payload $(C_j, \|\mathbf{x}_j\|)$ and the quantization payload η_j. All other components of the ADS and its construction are unchanged from Section V-A.

Definition 6.1 (Quantization-aware Graph Node ADS)**.** For each graph node v_i in the index $\mathcal{I}$, the ADS g_i contains: (i) ID_i, the identifier of the current node v_i; (ii) $\{(ID_j, C_j, \|\mathbf{x}_j\|, \eta_j)\}_{\forall v_j\in Adj_i}$, the identifiers, commitments, norms, and quantization payloads of all immediate neighbors of v_i in $\mathcal{I}$. The hash of g_i is defined as $H_i^g = \mathbb{H}(ID_i \,|\, \{ID_j \,|\, H_j^{exa} \,|\, H_j^{qua}\}_{\forall v_j\in Adj_i})$, where $H_j^{exa} = \mathbb{H}(C_j \,|\, \|\mathbf{x}_j\|)$ is the hash of the exact-distance payload of v_j, and $H_j^{qua} = \mathbb{H}(\eta_j)$ is the hash for the quantization payload.

Algorithm 3: Quantization-aware Verifiable Search

1 **Function** `QuantAwareSearch`$(Q=(k,\mathbf{x}_q),\mathcal{I},\mathcal{A},\sigma)$**:**
2 $PriorityQueue \leftarrow \{(v_{\text{ep}},\delta_{\text{ep}})\}$;
3 $V_{\text{merkle}} \leftarrow \{v_{\text{ep}}\}$; $V_{\text{exa}} \leftarrow \{v_{\text{ep}}\}$; $V_{\text{qua}} \leftarrow \varnothing$;
4 **while** $(v_i,\delta_i) \leftarrow PriorityQueue$.dequeue() **do**
5 Update result set R with v_i and stop if there are already k closer results;
6 $V_{\text{merkle}} \leftarrow V_{\text{merkle}} \cup \{v_i\}$;
7 **foreach** $v_j \in Adj_i$ **do**
8 $\delta_{\text{th}} \leftarrow PriorityQueue$.threshold(); $LB_j \leftarrow$ `QuantLB`$(\eta_j,\mathbf{x}_q)$;
9 **if** $LB_j > \delta_{\text{th}}$ **then**
10 $V_{\text{qua}} \leftarrow V_{\text{qua}} \cup \{v_j\}$; **continue**;
11 $\delta_j \leftarrow$ `ExactDist`$(\mathbf{x}_j,\mathbf{x}_q)$; $V_{\text{exa}} \leftarrow V_{\text{exa}} \cup \{v_j\}$;
12 $PriorityQueue$.enqueue$((v_j,\delta_j))$;
13 $\pi \leftarrow$ `ProveMerkle`$(V_{\text{merkle}},\mathcal{A})$;
14 $\rho_{agg} \leftarrow$ `ProveAggIPA`$(\mathbf{x}_q,\{\mathbf{x}_i\}_{v_i\in V_{\text{exa}}})$;
15 $VO \leftarrow (\{g_i^{op}\}_{v_i\in V_{\text{merkle}}},\pi,\sigma,\{IP_i\}_{v_i\in V_{\text{exa}}},\rho_{agg})$;
16 **return** (R,VO).

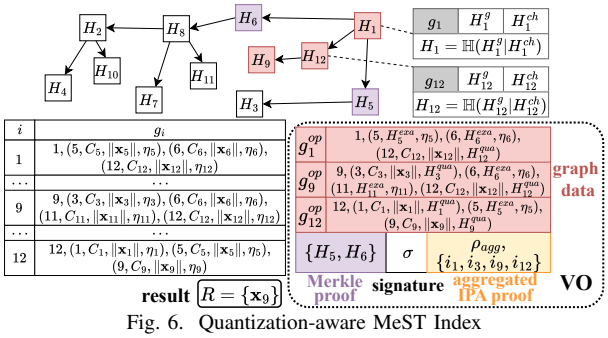


Fig. 6. Quantization-aware MeST Index

Verifiable Search. During query processing, the SP executes the quantization-aware graph search in Algorithm 3. The search follows the same priority-queue-guided traversal as the original graph search. When a node v_i is dequeued and expanded, the SP adds it to V_{merkle} and scans each neighbor $v_j \in Adj_i$. It first obtains the current search threshold δ_{th} and computes the lower bound LB_j from η_j and $\mathbf{x}_q$ (Line 8). If $LB_j > \delta_{\text{th}}$, then v_j is pruned without exact distance computation and recorded in the quantization-pruned node set V_{qua} (Line 10). Otherwise, the SP computes the exact distance δ_j, adds v_j to the distance-evaluated node set V_{exa}, and inserts it into the priority queue (Lines 11–12). If a node is encountered multiple times, membership in V_{exa} takes precedence: once its exact distance is evaluated, the node is excluded from V_{qua}. Then, for each $v_i \in V_{\text{merkle}}$, the SP materializes a node opening g_i^{op} that discloses only the payload required for each exposed neighbor. Distance-evaluated neighbors reveal $(C_j, \|\mathbf{x}_j\|)$, while quantization-pruned neighbors reveal η_j. The unopened component is represented by its digest. The SP then generates an aggregated IPA proof for the nodes in V_{exa} and constructs the VO.

Client-side verification is similar to that described in Section V-B, except that the distance-evaluated and quantization-pruned branches are handled separately. The client first reconstructs the graph node digests from $\{g_i^{op}\}_{v_i\in V_{\text{merkle}}}$ and then reconstructs the root hash. For nodes in V_{exa}, it verifies ρ_{agg} and recovers their authenticated distances. For each $v_j \in V_{\text{qua}}$, it computes LB_j from the opened η_j and checks $LB_j > \delta_{\text{th}}$ at the same replay point. Finally, the client replays the search to verify the ANN result.

Example. *Consider the search in Figure 2b. Suppose* $V_{\text{qua}} = \{v_5, v_6, v_{11}\}$ *is the quantization-pruned set, while the remaining visited nodes* $V_{\text{exa}} = \{v_1, v_3, v_9, v_{12}\}$ *are evaluated using exact distances. As shown in Figure 6, the node openings* $g_1^{op}, g_9^{op}, g_{12}^{op}$ *are included in the VO. Within each node opening, every exposed neighbor is disclosed according to the branch taken during the quantization-aware search. For example,* $6, H_6^{exa}, \eta_6$ *and* $9, C_9, \|\mathbf{x}_9\|, H_9^{qua}$ *are exposed differently. During verification, the client reconstructs the root and verifies the signature* σ *and proof* ρ_{agg}*. For each node in* V_{qua}*, the client recomputes* LB_j *from the opened quantization payload* η_j*. Finally, the client replays the search using exact distances for* V_{exa} *and lower bounds for* V_{qua}*.*

VII. Security Analysis

We first establish the computational knowledge soundness of IPA aggregation using the underlying IPA extractor [29], then use authenticated distances and deterministic replay to prove the execution integrity of VIDA and its variants. Complete proofs are provided in Appendices B and C.

Theorem 7.1 (Soundness of IPA Aggregation)**.** *Let* Adv *be a PPT adversary that produces a valid aggregated IPA proof for the relation* $\langle \mathbf{x}_{agg}, \mathbf{x}_q \rangle = IP_{agg}$ *with non-negligible probability. There exists a PPT extractor* $\mathcal{E}$ *that extracts the witnesses* $\{\mathbf{x}_i\}_{i=1}^n$ *such that* $\langle \mathbf{x}_i, \mathbf{x}_q \rangle = IP_i$ *for all* $i \in \{1,\ldots,n\}$*.*

Proof Sketch. By rewinding the prover at the Fiat–Shamir challenge [30], $\mathcal{E}$ obtains accepting transcripts for n distinct nonzero challenges. The IPA extractor yields an aggregated witness per transcript. Solving the resulting invertible Vandermonde system recovers the individual witnesses. Commitment homomorphism and binding tie each $\mathbf{x}_i$ to C_i, while inner-product linearity gives $\langle \mathbf{x}_i, \mathbf{x}_q \rangle = IP_i$. □

We next consider execution integrity under Definition 2.1, including the topology-aware and quantization-aware variants.

Theorem 7.2. *VIDA and its variants are secure with respect to Definition 2.1, assuming the underlying cryptographic primitives, including IPA aggregation, are secure.*

Proof Sketch. An accepted forgery must either alter the authenticated data or deviate from the designated search. Altering graph structure or metadata requires a hash collision or signature forgery; changing a committed vector violates binding; and falsifying an authenticated distance contradicts Theorem 7.1. Otherwise, replay uses genuine graph data and distances: omitting a required node causes rejection, while deterministic replay yields the honest result. MeST preserves this argument through authentication against the signed root. For the quantization-aware variant, pruning is checked using authenticated lower-bound payloads at the same replay threshold. Under the quantization guarantee, a valid pruning step, or its replacement by an exact evaluation, preserves the effective search state. Thus, an incorrect result can be accepted only with negligible probability. □

TABLE II
DATASET STATISTICS

Dataset	Size	Dimension	Query Size	Type
OpenAI3072 [21]	999,000	3,072	1,000	Text
WIT [32]	6,476,255	2,048	1,000	Image
LAION [33]	200,000,000	512	1,000	Image
Wikipedia [34]	106,712,786	1,024	1,000	Text
Msmarco [35]	113,520,750	1,024	1,677	Text
Cluster	10,000,000	4,096	1,000	Synthetic

VIII. EVALUATION

A. Experiment Settings

Datasets and Query Workload. We conduct extensive experiments on six datasets with diverse sizes, dimensions, and data types, as summarized in Table II. The synthetic Cluster dataset is generated following [31]. Regarding query workloads, we use the official query sets when available. For datasets without a predefined query set, we randomly sample 1,000 vectors from the base data to serve as queries. Unless otherwise specified, all experiments use Euclidean distance as the similarity metric and set the result size to $k = 100$. The target recall for each dataset follows the varying-k experiment.

Metrics. In our experiments, we primarily focus on end-to-end latency, defined as the total time elapsed from the client sending a query until completion of the verification process. We report recall to evaluate search accuracy. Furthermore, to assess index construction overhead and storage efficiency, we measure the construction time and the size of the ADS.

Main Algorithms for Evaluation. We evaluate three base methods and their quantization-aware variants. **Strawman** employs a standard MHT over the graph and transmits raw vectors for distance verification. **ANNProof** [18] improves Strawman with a sharded MHT, while retaining raw-vector-based verification. **VIDA*** is the full topology-aware instantiation of VIDA described in Section V, combining aggregated IPA with MeST. Their quantization-aware variants, **Strawman-Q**, **ANNProof-Q**, and **VIDA*-Q**, apply the quantization-aware search in Section VI, using authenticated quantization payloads for pruned candidates and their respective original mechanisms for exact-distance verification.

We use HNSW [9] as the underlying ANN search index, unless otherwise specified. For the HNSW index configuration, we set the construction parameter $ef_{\text{construction}}$ to 500 across all datasets. The maximum number of links per node, M, is set to 32. During the query phase, we vary ef_{search} to achieve different trade-offs between recall and latency. For IPA, we instantiate the protocol using the lightweight Bulletproofs [22] (a variation) together with the Springproofs [23] framework. For quantization-aware variants, we use RaBitQ [28] on top of the corresponding graph index.

Environment. We implement the proposed schemes and baselines in the Rust programming language. The source codes are available at https://github.com/hkbudb/auth-ann. For the cryptographic primitives, the IPA is implemented via Bulletproofs [36] using the Ristretto255 curve for group operations. All experiments were conducted on a server equipped with Dual 28-core Intel Xeon Gold 6330 @ 2.0GHz CPUs and 2TB RAM. To simulate real-world outsourced database scenarios, we limit the network bandwidth using the toxiproxy utility [37]. The default network bandwidth is set to 300 Mbps.

B. Search Performance

1) Latency under Varying Recall Targets: We first evaluate the trade-off between end-to-end query latency and recall. Figure 7 compares the methods across recall targets on six datasets. For Msmarco, we use NSG [10] as the underlying ANN algorithm to evaluate our approach's generality beyond HNSW. We also include Strawman and its quantization-aware variant Strawman-Q as reference baselines. As shown in Figure 7, Strawman(-Q) and ANNProof(-Q) show very similar latency trends. We therefore omit Strawman and use ANNProof as the baseline in subsequent experiments.

Overall, VIDA* consistently outperforms Strawman and ANNProof across all datasets, and the advantage becomes more pronounced in high-recall settings. This trend is especially evident on high-dimensional OpenAI3072. At the highest recall target, VIDA* achieves a 6.6× speedup over ANNProof, while VIDA*-Q achieves a 3.0× speedup over ANNProof-Q. These results are consistent with our analysis. Achieving higher recall requires increasing the search parameter, *e.g.*, ef_{search} for HNSW, which increases the number of nodes visited during graph traversal. For raw-vector-based baselines, the transmission cost grows with the number of visited nodes and becomes the dominant contributor to end-to-end latency. In contrast, VIDA* replaces raw-vector transmission with compact commitments and aggregated IPA proofs, and further uses MeST to place neighboring graph nodes close together in the authentication tree. This reduces both distance-authentication cost and structural proof size. For instance, on OpenAI3072, VIDA* reduces the transfer cost by 99.2% compared with ANNProof at the highest recall target, while VIDA*-Q reduces it by 82.5% compared with ANNProof-Q. Quantization-aware variants further reduce latency by reducing the number of candidates requiring exact-distance authentication. Compared with VIDA*, VIDA*-Q achieves additional speedups of 1.9×, 1.7×, and 1.4× on OpenAI3072, WIT, and Wikipedia, respectively, confirming that quantization complements our cryptographic and topology-aware optimizations.

2) Latency under Varying k: We vary k from 25 to 400 and tune the search parameters to maintain each dataset's target recall. Figure 8 decomposes latency on OpenAI3072, WIT, and LAION into server-side verifiable search (Server), network transfer (Transfer), and client-side verification (Client). Results for the other datasets are in Appendix D. Across all six datasets, VIDA* and VIDA*-Q consistently outperform their corresponding baselines across all k values, achieving up to 6.5× and 3.4× speedups over ANNProof and ANNProof-Q, respectively. The gains are especially pronounced on high-dimensional datasets. On OpenAI3072 with $k = 400$, ANNProof and ANNProof-Q incur 6,109 ms and 1,506 ms, respectively, whereas VIDA*-Q requires only 519 ms and reduces transfer cost by 82.5% over ANNProof-Q. This confirms that our quantization-aware authentication further re-

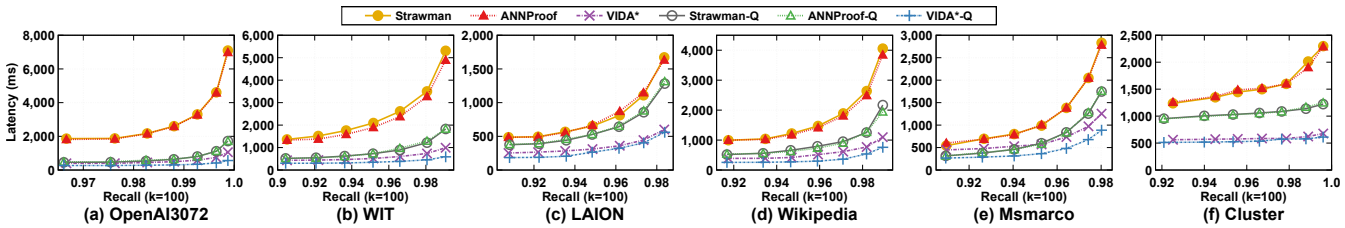


Fig. 7. Latency on Different Recalls

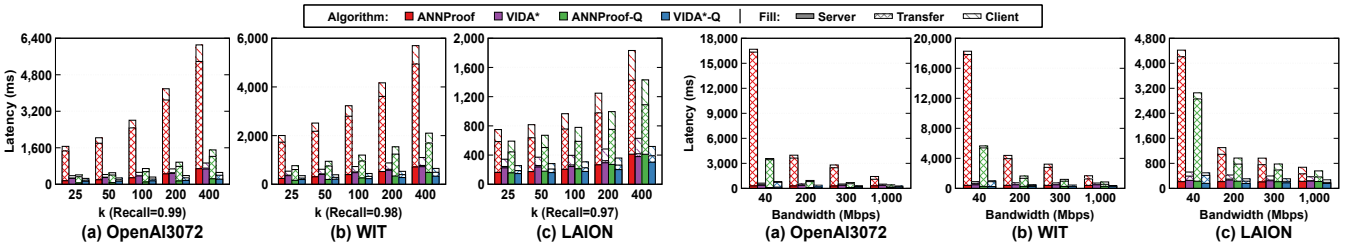


Fig. 8. Latency of Varying k

Fig. 9. Latency on Different Network Settings

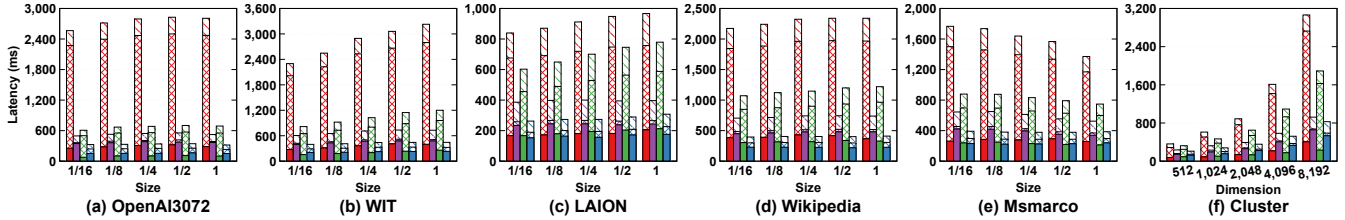


Fig. 10. Scalability Test

duces communication overhead, even against the quantization-aware baseline.

Latency generally increases with k as more nodes are visited and larger VOs are generated, but the growth is nonlinear. At a fixed recall, small-k queries may still require large search parameters, so node visits do not decrease proportionally with k. This effect makes the raw-vector-based baselines costly even at small k, whereas our schemes grow more slowly. On OpenAI3072, when k increases from 25 to 400, the latency of VIDA*-Q increases by only 2.0×, compared with 2.5× for VIDA*, 3.7× for ANNProof, and 3.6× for ANNProof-Q. We also test smaller k at lower recall targets on this dataset (see Section E-B).

3) Latency under Varying Network Bandwidth: We next evaluate performance under different network conditions by simulating four bandwidth profiles [38]: 4G (40 Mbps), 5G (200 Mbps), Broadband (300 Mbps), and Server (1,000 Mbps). Figure 9 reports results on three datasets, while results for the other datasets are in Appendix D. ANNProof and ANNProof-Q are highly sensitive to bandwidth degradation. Their latency increases sharply as available bandwidth decreases, indicating that large-scale data transmission remains the dominant bottleneck. For example, on OpenAI3072, when moving from Server bandwidth to 4G, the latency of ANNProof and ANNProof-Q increases by 11.7× and 8.5×, respectively.

In contrast, VIDA* and VIDA*-Q remain much more robust under bandwidth-limited settings. On OpenAI3072, the latency of VIDA* increases by only 1.2× when moving from 1,000 Mbps to 40 Mbps, while VIDA*-Q increases by 2.8×. Under the most constrained 4G setting, ANNProof and ANNProof-Q incur 16.7 s and 3.6 s latency, respectively, whereas VIDA* and VIDA*-Q reduce the latency to 0.63 s and 0.82 s. Compared with ANNProof-Q, VIDA*-Q reduces the transfer cost by 82.8% on OpenAI3072 and by 87.3% on WIT. These results show that compact distance authentication and topology-aware proofs substantially reduce the communication bottleneck, while quantization-aware authentication remains effective against its corresponding quantized baseline.

C. Scalability and Efficiency

We next evaluate the efficiency of the proposed schemes from three perspectives: scalability, offline construction overhead, and the contribution of the core optimization mechanisms. Unless otherwise specified, we fix the search parameters across different methods.

1) Scalability Analysis: We first study how query latency varies with dataset cardinality. To this end, we construct subsets of different sizes by randomly sampling from five real-world datasets. As shown in Figure 10(a)-(e), for datasets such as OpenAI3072 and Wikipedia, query latency increases only moderately as the dataset size grows, which is consistent with the expected logarithmic scaling behavior of graph-based indexes. An exception is the Msmarco dataset. For this sparse and high-dimensional dataset, random sampling degrades graph navigability, forcing the search procedure to visit more nodes to maintain the target recall.

We then evaluate the impact of vector dimensionality using the synthetic Cluster dataset. We fix the dataset size and vary the vector dimension d from 512 to 8,192. Figure 10(f) shows that ANNProof is highly sensitive to vector dimensionality. Its latency increases by 8.5×, while ANNProof-Q still increases by 5.8×. In contrast, VIDA* and VIDA*-Q increase by

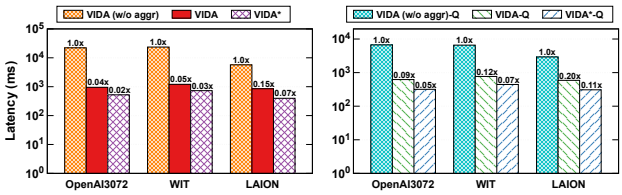


Fig. 11. Ablation Study

only 3.9× and 3.9×, respectively. This result suggests that our IPA-based distance authentication significantly reduces the dependence of verification cost on vector dimensionality, making our schemes particularly well suited for modern high-dimensional embedding workloads. We also evaluate query latency at lower dimensions (see Section E-B).

2) Offline Construction: We evaluate offline index initialization in terms of construction time and ADS size. Relative to their corresponding baselines, VIDA* and VIDA*-Q require 5.5× and 5.4× the construction time on average, with ADS size increases of 116.1% and 75.4%, respectively. This one-time cost is paid by the data owner in exchange for the online latency and communication reductions. Detailed results and analysis are in Section E-A.

3) Ablation on Core Optimizations: Finally, we evaluate two core optimizations: proof aggregation and the topology-aware ADS. We compare three variants. VIDA (w/o aggr) uses the same MHT structure as Strawman, but replaces raw-vector replay with individual IPA proofs for each visited node and applies batch verification [29]. VIDA further replaces individual IPA proofs with a single aggregated proof. VIDA* further augments VIDA with MeST, our topology-aware ADS. We also include their quantization-aware versions. As shown in Figure 11, proof aggregation is essential. Since VIDA (w/o aggr) and VIDA use the same MHT structure, their gap isolates the benefit of aggregation. VIDA reduces total latency over VIDA (w/o aggr) by 91.9% on average, while VIDA-Q reduces latency over VIDA (w/o aggr)-Q by 86.3%. The improvement is especially significant on OpenAI3072, where aggregation provides 23.3× and 10.8× speedups in the non-quantized and quantization-aware settings, respectively. The gap between VIDA and VIDA* further isolates the benefit of MeST: VIDA* reduces total latency by 46.0% over VIDA on average, and VIDA*-Q reduces latency by 46.3% over VIDA-Q. These results confirm that aggregation removes the dominant per-node IPA bottleneck, while MeST further reduces structural authentication overhead.

IX. Related Work

ANN Search. ANN search algorithms are broadly categorized into hash-based [7], inverted-file-based [8], and graph-based methods [39]. Currently, graph-based methods are widely recognized for achieving the best trade-off between search efficiency and recall [11]. These methods operate as a greedy best-first search on a proximity graph. Based on graph topology and edge pruning strategies, graph-based ANN methods are categorized into hierarchical (*e.g.*, HNSW [9], HVS [40]) and flat structures (*e.g.*, NSG [10], DPG [41]). Recent research has expanded graph indexes to diverse environments, including disk-resident designs for massive datasets (*e.g.*, DiskANN [42]), hardware-aware layouts for GPUs (*e.g.*, CAGRA [43]), and metadata-filtered searches [44]. Despite these optimizations, their core search paradigm remains reliant on sequential graph traversals.

Verifiable Query Processing. Verifiable query processing over structured databases has been widely studied using authenticated data structures (ADS) [15] or zero-knowledge proofs [16], [17] to ensure query integrity across various query types [45]–[48]. When extending verifiable queries to vector data, existing works mainly target exact kNN search. These works generally leverage spatial partitioning ADS (*e.g.*, MR-trees [49], Voronoi diagrams [50]) to ensure result integrity by rigorously proving that the unsearched space cannot contain better candidates. Regarding ANN search, however, this exact verification paradigm is fundamentally incompatible: high dimensionality renders spatial geometric proofs prohibitively expensive, while enforcing exactness contradicts the efficiency goal of approximate search. Consequently, verifiable ANN should shift its focus from strictly verifying the final exact result to guaranteeing *execution integrity*, ensuring that the search algorithm is executed correctly and faithfully.

For graph-based ANN, ANNProof [18] and Wang et al. [19] use MHTs and raw-vector replay to verify HNSW and HCNNG searches, respectively. Wang et al. further reduce the number of probed nodes through HCNNG-specific optimizations. V3DB [51] and zkRAG [52] verify IVF-PQ and HNSW retrieval, respectively, using succinct zero-knowledge proofs that hide private database and index information. However, generating such proofs incurs substantial computational overhead. For example, zkRAG reports a proof generation time of approximately 26 seconds per query on SIFT1M. In contrast, VIDA targets general graph-based ANN methods. It integrates vector commitments and aggregated IPA to authenticate distances without transmitting raw vectors, and uses topology-aware MeST to reduce structural proof overhead. Separately, privacy-preserving ANN methods protect vector values and access patterns from service providers [53]–[57].

X. Conclusion

In this paper, we presented VIDA for efficient verifiable graph-based ANN search in outsourced vector databases. VIDA combines vector commitments with aggregated IPA proofs to authenticate distances without transmitting raw vectors. Furthermore, we developed MeST, a topology-aware Merkle spanning tree index that aligns structural verification paths with graph traversal sequences to minimize required Merkle proofs. We also adapted VIDA to quantization-aware graph search to avoid unnecessary exact-distance proofs. Our formal security analysis and experimental evaluations demonstrated that the proposed approach is secure and highly scalable. It delivers up to a 6.6× speedup in end-to-end latency while reducing data transfer time by 99.2% compared with state-of-the-art methods.

AI USAGE DECLARATION

Generative AI tools were used only for polishing the writing and assisting with code implementation.

REFERENCES

[1] "Milvus," https://milvus.io, [Accessed: 2026-6-1].

[2] "Qdrant," https://qdrant.tech, [Accessed: 2026-6-1].

[3] "Weaviate," https://weaviate.io, [Accessed: 2026-6-1].

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, 2020.

[5] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *ACM Comput. Surv.*, vol. 52, no. 1, pp. 1–38, 2019.

[6] V. Karpukhin, B. Oguz, S. Min, P. S. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.

[7] Y. Liu, J. Cui, Z. Huang, H. Li, and H. T. Shen, "SK-LSH: An efficient index structure for approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 7, no. 9, pp. 745–756, 2014.

[8] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2010.

[9] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2018.

[10] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proc. VLDB Endow.*, vol. 12, no. 5, pp. 461–474, 2019.

[11] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *VLDB J.*, vol. 33, no. 5, pp. 1591–1615, 2024.

[12] Y. Song, X. Zhou, C. S. Jensen, and J. Xu, "Vector search for the future: From memory-resident, static heterogeneous storage, to cloud-native architectures," in *Proc. ACM Manage. Data*, 2026.

[13] Z. Niu, X. Tian, X. Peng, and X. Chen, "BlendHouse: A cloud-native vector database system in ByteHouse," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2025, pp. 4332–4345.

[14] N. Upreti, H. V. Simhadri, H. S. Sundar, K. Sundaram, S. Boshra, B. Perumalswamy, S. Atri, M. Chisholm, R. R. Singh, G. Yang *et al.*, "Cost-effective, low latency vector search with azure cosmos db," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5166–5183, 2025.

[15] Y. Zhang, J. Katz, and C. Papamanthou, "IntegriDB: Verifiable SQL for outsourced databases," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2015, pp. 1480–1491.

[16] Y. Zhang, D. Genkin, J. Katz, D. Papadopoulos, and C. Papamanthou, "vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases," in *Proc. IEEE Symp. Secur. Privacy (SP)*. IEEE, 2017, pp. 863–880.

[17] X. Li, C. Weng, Y. Xu, X. Wang, and J. Rogers, "ZKSQL: Verifiable and efficient query evaluation with zero-knowledge proofs," *Proc. VLDB Endow.*, vol. 16, no. 8, 2023.

[18] L. Lu, Z. Wen, Y. Yuan, Q. He, J. Chen, and Z. Liu, "ANNProof: Building a verifiable and efficient outsourced approximate nearest neighbor search system on blockchain," *Future Gener. Comput. Syst.*, vol. 156, pp. 206–220, 2024.

[19] C. Wang, J. Zhang, X. Liu, K. Wei, and B. Feng, "Verifiable graph-based approximate nearest neighbor search," in *Proc. Int. Conf. Adv. Data Min. Appl.* Springer, 2024, pp. 3–17.

[20] R. C. Merkle, "A certified digital signature," in *Proc. CRYPTO*. Springer, 1989, pp. 218–238.

[21] "OpenAI3072," https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M, [Accessed: 2026-6-1].

[22] C. Gentry, S. Halevi, and V. Lyubashevsky, "Practical non-interactive publicly verifiable secret sharing with thousands of parties," in *Proc. EUROCRYPT*. Springer, 2022, pp. 458–487.

[23] J. Zhang, M. Su, X. Liu, and G. Wang, "Springproofs: Efficient inner product arguments for vectors of arbitrary length," in *Proc. IEEE Symp. Secur. Privacy (SP)*. IEEE, 2024, pp. 3147–3164.

[24] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. CRYPTO*. Springer, 1991, pp. 129–140.

[25] R. Henry and I. Goldberg, "Batch proofs of partial knowledge," in *Proc. Appl. Cryptogr. Netw. Secur.* Springer, 2013, pp. 502–517.

[26] M. Bellare and P. Rogaway, "Random oracles are practical: A paradigm for designing efficient protocols," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 1993, pp. 62–73.

[27] W. Zhou, Y. Lu, H. Li, and Q. Tian, "Scalar quantization for large scale image search," in *Proc. ACM Int. Conf. Multimedia (MM)*. ACM, 2012, pp. 169–178.

[28] J. Gao and C. Long, "RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search," *Proc. ACM Manage. Data*, vol. 2, no. 3, pp. 1–27, 2024.

[29] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symp. Secur. Privacy (SP)*. IEEE, 2018, pp. 315–334.

[30] J. Bootle, A. Cerulli, P. Chaidos, J. Groth, and C. Petit, "Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Tech.* Springer, 2016, pp. 327–357.

[31] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 1964–1978, 2021.

[32] K. Srinivasan, K. Raman, J. Chen, M. Bendersky, and M. Najork, "WIT: Wikipedia-based image text dataset for multimodal multilingual machine learning," in *Proc. ACM SIGIR Int. Conf. Res. Dev. Inf. Retr.*, 2021, pp. 2443–2449.

[33] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki, "LAION-400M: Open dataset of clip-filtered 400 million image-text pairs," *arXiv preprint arXiv:2111.02114*, 2021.

[34] "Wikipedia," https://huggingface.co/datasets/Cohere/wikipedia-2023-11-embed-multilingual-v3, [Accessed: 2026-6-1].

[35] "Msmarco," https://huggingface.co/datasets/Cohere/msmarco-v2.1-embed-english-v3, [Accessed: 2026-6-1].

[36] "Bulletproofs," https://github.com/zkcrypto/bulletproofs, [Accessed: 2026-6-1].

[37] "Toxiproxy," https://github.com/Shopify/toxiproxy, [Accessed: 2026-6-1].

[38] X. Yang, H. Lin, Z. Li, F. Qian, X. Li, Z. He, X. Wu, X. Wang, Y. Liu, Z. Liao, D. Hu, and T. Xu, "Mobile access bandwidth in practice: Measurement, analysis, and implications," in *Proc. ACM SIGCOMM Conf.*, 2022, pp. 114–128.

[39] I. Azizi, K. Echihabi, and T. Palpanas, "Graph-based vector search: An experimental evaluation of the state-of-the-art," *Proc. ACM Manage. Data*, vol. 3, no. 1, pp. 1–31, 2025.

[40] K. Lu, M. Kudo, C. Xiao, and Y. Ishikawa, "HVS: Hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 15, no. 2, pp. 246–258, 2021.

[41] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1475–1488, 2019.

[42] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[43] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, "Cagra: Highly parallel graph construction and approximate nearest neighbor search for GPUs," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2024, pp. 4236–4247.

[44] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of filtered approximate nearest neighbor search over the vector-scalar hybrid data," *arXiv preprint arXiv:2505.06501*, 2025.

[45] C. Zhang, C. Xu, H. Wang, J. Xu, and B. Choi, "Authenticated keyword search in scalable hybrid-storage blockchains," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2021, pp. 996–1007.

[46] H. Wang, C. Xu, C. Zhang, J. Xu, Z. Peng, and J. Pei, "vChain+: Optimizing verifiable blockchain boolean range queries," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2022, pp. 1927–1940.

[47] P. Sun, L. Zhang, J. Liu, C. Tang, and J. Wang, "E^{3}FS: Efficient, secure, and verifiable fuzzy search with data updates in hybrid-storage

blockchains," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2025, pp. 3890–3903.
[48] Z. Yao, T. Li, J. Xin, Y. Li, C. Wang, Z. Wang, D. Srivastava, and C. S. Jensen, "VGQ: Enabling verifiable graph queries on blockchain systems," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2025, pp. 3602–3614.
[49] C. Zhang, C. Xu, J. Xu, and B. Choi, "Distributed kNN query authentication," in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*. IEEE, 2018, pp. 167–176.
[50] N. Cui, K. Qian, T. Cai, J. Li, X. Yang, J. Cui, and H. Zhong, "Towards multi-user, secure, and verifiable kNN query in cloud database," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9333–9349, 2023.
[51] Z. Qiu, W. Qu, J. Zhang, and B. Yuan, "V3DB: Audit-on-demand zero-knowledge proofs for verifiable vector search over committed snapshots," *Proc. VLDB Endow.*, vol. 19, no. 10, pp. 2604–2616, 2026.
[52] Y. Jiang, X. Yang, X. Liu, Y. Guo, and J. Zhang, "zkRAG: Efficiently proving RAG retrieval in zero knowledge," *Cryptology ePrint Archive*, no. 2026/709, 2026. [Online]. Available: https://eprint.iacr.org/2026/709
[53] J. Zhu, L. Patel, M. Zaharia, and R. A. Popa, "Compass: Encrypted semantic search with high accuracy," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2025, pp. 915–938.
[54] H. Chen, I. Chillotti, Y. Dong, O. Poburinnaya, I. Razenshteyn, and M. S. Riazi, "SANNS: Scaling up secure approximate k-nearest neighbors search," in *Proc. USENIX Secur. Symp.*, 2020, pp. 2111–2128.
[55] J. Li, Z. Huang, M. Zhang, C. Hong, J. Liu, T. Wei, and W. Chen, "Panther: Private approximate nearest neighbor search in the single server setting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2025, pp. 365–379.
[56] M. Zhou, E. Shi, and G. Fanti, "Pacmann: Efficient private approximate nearest neighbor search," *Cryptology ePrint Archive*, 2024.
[57] Y. Liu, Y. Zhang, J. Xie, H. Li, J. X. Yu, and J. Cui, "Privacy-preserving approximate nearest neighbor search on high-dimensional data," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*. IEEE, 2025, pp. 3017–3029.

APPENDIX A
MEST FOR MULTI-LAYERED GRAPHS

In graph-based ANN search, a prominent class of algorithms employs multi-layered graph structures [9], which effectively function as skip lists to accelerate query processing. These indexes organize the graph into a hierarchy, enabling the query to navigate from sparser upper layers all the way down to the dense base layer. To further exploit this hierarchical topology, we extend MeST with a layer-aware design that aligns the Merkle spanning tree with the existing graph layers.

ADS Construction. To construct a unified spanning tree across multiple layers, the DO first consolidates the subgraphs from all layers into a single unified graph. By flattening the hierarchical structure, the DO can invoke Algorithm 1 on the resulting graph to build a distance-aware spanning tree. This flattening step ensures that the spanning tree captures the skip-list jumps provided by the original hierarchical index. Since the ANN search may traverse neighbors within a specific layer at any given time, we introduce a Multi-layered Graph Node ADS to maintain this structural granularity.

Definition A.1 (Multi-layered Graph Node ADS)**.** For each graph node v_i in the multi-layered index $\mathcal{I}$, its corresponding ADS g_i consists of (i) ID_i: the identifier of the current node v_i; (ii) multiple layer-specific neighbors: for each layer $0 \le l \le L_i$, a set $g_i^{(l)} = \{(ID_j, C_j, \|\mathbf{x}_j\|)\}_{\forall v_j \in Adj_i^{(l)}}$ containing the identifiers, commitments, and norms of the immediate neighbors of v_i at layer l. Here, $Adj_i^{(l)}$ denotes the immediate neighbors at that layer. The hash of g_i is defined as $H_i^g = \mathbb{H}(ID_i \mid H_i^{(0)} \mid \cdots \mid H_i^{(L_i)})$, where $H_i^{(l)}$ is the hash for the neighbors at the l-th layer, defined as $H_i^{(l)} = \mathbb{H}(g_i^{(l)}) = \mathbb{H}(\{ID_j \mid C_j \mid \|\mathbf{x}_j\|\}_{\forall v_j \in Adj_i^{(l)}})$.

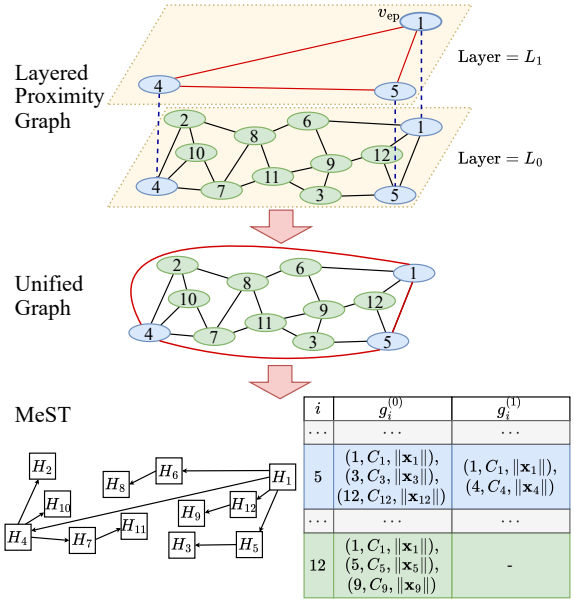


Fig. 12. An Example on a Multi-layered Graph

The remainder of the ADS structure and its construction algorithm are identical to those described in Section V-A.

Example. *Consider the two-layered proximity graph in Figure 12. MeST first forms a unified graph by taking the union of edges across all layers, then builds the spanning tree over this unified graph. For example, starting from the entry point v_1, the spanning tree algorithm reaches node v_4 through the adjacency list of v_1 on layer L_1, and reaches nodes v_5, v_6, and v_{12} through the adjacency list of v_1 on layer L_0. Based on the spanning tree, MeST is constructed in a layer-aware manner, where the graph ADS of each node is organized into $g_i^{(1)}$ and $g_i^{(0)}$. For instance, $g_5^{(1)}$ contains $(ID_1, C_1, \|\mathbf{x}_1\|)$ and $(ID_4, C_4, \|\mathbf{x}_4\|)$, corresponding to its neighbors v_1 and v_4 on layer L_1, while v_{12} only has $g_{12}^{(0)}$ because it does not appear on layer L_1.*

Verifiable Search. In the multi-layered graph index, the search state is determined not only by the visited node, but also by the layer at which the node is accessed. To avoid disclosing redundant neighborhood information, we adopt a layer-specific disclosure strategy. Concretely, if a node v_i is accessed at layer l during the search, the SP reveals only the neighborhood payload $g_i^{(l)}$ for that layer. For any unaccessed layer l', the VO contains only the corresponding digest $H_i^{(l')}$ rather than the full neighbor information. Therefore, the VO materializes only the layer-specific data actually used during the search, while the remaining layers are authenticated via their hashes. During verification, the client replays the designated ANN algorithm from the top layer all the way down to the base layer. If the replay attempts to access the neighborhood information of a node v_i on a layer whose payload is not disclosed in the VO, the verification fails immediately. In this way, MeST avoids transmitting unused layer-specific graph information.

APPENDIX B
SOUNDNESS OF IPA AGGREGATION

Our aggregation mechanism relies on the computational knowledge soundness of the underlying IPA. Intuitively, this property guarantees that a prover cannot produce a valid proof without actually knowing the corresponding witness (*i.e.*, the vector value).

Lemma B.1 (Soundness of Single IPA)**.** *The single IPA protocol used in our scheme satisfies computational knowledge soundness. That is, for any PPT adversary* Adv *producing a valid proof with non-negligible probability, there exists a PPT extractor $\mathcal{E}_{IPA}$ that extracts the witness $\mathbf{x}_i$ in the relation $\langle \mathbf{x}_i, \mathbf{x}_q \rangle = IP_i$.*

Proof. This claim follows from the soundness of the underlying IPA protocol, such as Bulletproofs [29]. □

Theorem 7.1 (Soundness of IPA Aggregation)**.** *Let* Adv *be a PPT adversary that produces a valid aggregated IPA proof for the relation $\langle \mathbf{x}_{agg}, \mathbf{x}_q \rangle = IP_{agg}$ with non-negligible probability. There exists a PPT extractor $\mathcal{E}$ that extracts the witnesses $\{\mathbf{x}_i\}_{i=1}^n$ such that $\langle \mathbf{x}_i, \mathbf{x}_q \rangle = IP_i$ for all $i \in \{1, \ldots, n\}$.*

Proof. The proof proceeds by constructing an extractor $\mathcal{E}$ that utilizes the adversary Adv as a black box.

Transcript Generation via Rewinding. The aggregation protocol involves a random challenge scalar r derived via Fiat-Shamir. Let n be the number of aggregated instances. The extractor $\mathcal{E}$ runs Adv to obtain a valid transcript with an initial challenge r_1. By applying the General Forking Lemma [30], $\mathcal{E}$ rewinds Adv to the point of challenge generation and re-runs it multiple times with distinct random challenges $r_2, \ldots, r_n$. Assuming Adv succeeds with non-negligible probability ϵ, $\mathcal{E}$ obtains n valid transcripts corresponding to a set of distinct challenges $\{r_1, r_2, \ldots, r_n\}$. Note that while extending to n branches introduces a polynomial reduction loss, the success probability remains non-negligible for polynomial n.

Extraction of Aggregated Witness. For each transcript $\ell \in \{1, \ldots, n\}$, the underlying proof is a valid single IPA for the aggregated statement defined by the challenge r_ℓ:

$$C_{agg}^{(\ell)} = \sum_{i=1}^{n} (r_\ell)^i \cdot C_i, \quad IP_{agg}^{(\ell)} = \sum_{i=1}^{n} (r_\ell)^i \cdot IP_i \tag{2}$$

Since the underlying IPA protocol satisfies computational knowledge soundness, for each transcript ℓ, we can invoke its extractor $\mathcal{E}_{IPA}$ to extract a witness vector $\hat{\mathbf{x}}^{(\ell)}$ satisfying:

$$\texttt{Com}(\hat{\mathbf{x}}^{(\ell)}) = C_{agg}^{(\ell)} \quad \text{and} \quad \langle \hat{\mathbf{x}}^{(\ell)}, \mathbf{x}_q \rangle = IP_{agg}^{(\ell)} \tag{3}$$

Let $\{\mathbf{x}_i\}_{i=1}^n$ be the actual witnesses. We now show how to compute them. The relationship between the extracted aggregated witnesses and the individual witnesses can be modeled as a system of linear equations:

$$\begin{pmatrix} \hat{\mathbf{x}}^{(1)} \\ \hat{\mathbf{x}}^{(2)} \\ \vdots \\ \hat{\mathbf{x}}^{(n)} \end{pmatrix} = \begin{pmatrix} r_1 & r_1^2 & \ldots & r_1^n \\ r_2 & r_2^2 & \ldots & r_2^n \\ \vdots & \vdots & \ddots & \vdots \\ r_n & r_n^2 & \ldots & r_n^n \end{pmatrix} \begin{pmatrix} \mathbf{x}'_1 \\ \mathbf{x}'_2 \\ \vdots \\ \mathbf{x}'_n \end{pmatrix} \tag{4}$$

The matrix $\mathbf{M}$ is a Vandermonde matrix, which is invertible provided all r_ℓ are distinct and non-zero. The extractor $\mathcal{E}$ computes the unique solution $\{\mathbf{x}'_i\}_{i=1}^n = \mathbf{M}^{-1} \cdot \{\hat{\mathbf{x}}^{(\ell)}\}_{\ell=1}^n$.

Due to the homomorphic property of the commitment scheme, the value $\sum_{i=1}^n (r_\ell)^i \mathbf{x}'_i$ is a valid opening for $C_{agg}^{(\ell)}$. Under the computational binding property of the Pedersen commitment guaranteed by the Discrete Logarithm assumption, it is computationally infeasible for the adversary to open $C_{agg}^{(\ell)}$ to a value different from the linear combination of the original openings. Therefore, except with negligible probability, the solved values must be the true witnesses committed in C_i. Thus, we have $\mathbf{x}_i = \mathbf{x}'_i$.

Polynomial Analysis. Having extracted the valid witnesses $\mathbf{x}_i$ for the commitments, we now verify they satisfy the inner product relations. For every $\ell \in \{1, \ldots, n\}$, witness extraction gives:

$$\langle \hat{\mathbf{x}}^{(\ell)}, \mathbf{x}_q \rangle = IP_{agg}^{(\ell)} \tag{5}$$

Substituting the extracted linear combinations:

$$\left\langle \sum_{i=1}^{n} (r_\ell)^i \mathbf{x}_i, \mathbf{x}_q \right\rangle = \sum_{i=1}^{n} (r_\ell)^i IP_i \tag{6}$$

Since $\mathbf{x}_q$ is fixed and public, the inner product operator $\langle \cdot, \mathbf{x}_q \rangle$ is linear. We can rewrite the equation as:

$$\begin{aligned} &\sum_{i=1}^{n} (r_\ell)^i \langle \mathbf{x}_i, \mathbf{x}_q \rangle = \sum_{i=1}^{n} (r_\ell)^i IP_i \\ \implies &\sum_{i=1}^{n} (r_\ell)^i \left(\langle \mathbf{x}_i, \mathbf{x}_q \rangle - IP_i \right) = 0 \end{aligned} \tag{7}$$

Let $\Delta_i = \langle \mathbf{x}_i, \mathbf{x}_q \rangle - IP_i$. This defines a homogeneous linear system $\mathbf{M} \cdot \mathbf{\Delta} = \mathbf{0}$, where $\mathbf{\Delta} = (\Delta_1, \ldots, \Delta_n)^T$ and $\mathbf{M}$ is the same invertible Vandermonde matrix used for extraction.

$$\begin{pmatrix} r_1 & r_1^2 & \ldots & r_1^n \\ r_2 & r_2^2 & \ldots & r_2^n \\ \vdots & \vdots & \ddots & \vdots \\ r_n & r_n^2 & \ldots & r_n^n \end{pmatrix} \begin{pmatrix} \Delta_1 \\ \Delta_2 \\ \vdots \\ \Delta_n \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \tag{8}$$

Since $\mathbf{M}$ is non-singular, the only solution is the trivial solution:

$$\Delta_1 = \Delta_2 = \cdots = \Delta_n = 0 \tag{9}$$

This implies that $\langle \mathbf{x}_i, \mathbf{x}_q \rangle - IP_i = 0$, and thus $\langle \mathbf{x}_i, \mathbf{x}_q \rangle = IP_i$ for all $i \in \{1, \ldots, n\}$.

We have constructed an extractor $\mathcal{E}$ that succeeds in extracting valid witnesses for all constituent instances whenever the adversary succeeds. This proves that the aggregation protocol satisfies computational knowledge soundness. □

APPENDIX C
SECURITY OF VIDA

Theorem 7.2. *VIDA and its variants are secure with respect to Definition 2.1, assuming the underlying cryptographic primitives, including IPA aggregation, are secure.*

Proof. Let Adv be a PPT adversary that wins the experiment in Definition 2.1 with non-negligible probability. Namely, given a database $\mathcal{D}$ and a query Q, Adv outputs a query result R_{Real} and a verification object VO such that $\texttt{Verify}(Q, R_{\mathsf{Real}}, \text{VO}) = \texttt{accept}$, yet $R_{\mathsf{Real}} \neq R_{\mathsf{Ideal}}$,

where R_{Ideal} is the output of the designated deterministic ANN algorithm faithfully executed on the authentic index generated from $\mathcal{D}$. For the quantization-aware instantiation, the designated algorithm is the deterministic quantization-aware graph search designated by the DO.

We show that such an adversary must break at least one of the underlying cryptographic assumptions with non-negligible probability.

Integrity of the Local Subindex. During verification, the client reconstructs a local subgraph $\mathcal{I}'$ from the data carried in the VO. Let $\mathcal{I}$ denote the authentic ANN index generated during the setup phase. We analyze the integrity of $\mathcal{I}'$ from the following three aspects.

- *Structural Integrity.* If the structural information of any node in $\mathcal{I}'$ (*e.g.*, node identifier or adjacency list) differs from that in $\mathcal{I}$, yet still passes Merkle-root verification against the signed root digest, then Adv must either forge a signature or find a collision in $\mathbb{H}$ in order to produce a valid proof for altered data.
- *Data Integrity.* If the commitment C_i or the authenticated norm $\|\mathbf{x}_i\|$ associated with a node v_i in $\mathcal{I}'$ differs from the corresponding value in $\mathcal{I}$, while the verification still accepts, then again Adv must either forge a signature or find a hash collision. Moreover, if Adv claims that a commitment C_i opens to some vector $\mathbf{x}'_i \neq \mathbf{x}_i$, then this violates the computational binding property of the Pedersen commitment.
- *Distance Integrity.* If the recovered distance between a node vector and the query vector differs from the true distance induced by the vector committed in C_i, then Adv must have produced a valid aggregated IPA proof for an invalid inner-product relation. This contradicts the computational knowledge soundness of the underlying IPA protocol.

Therefore, except with negligible probability, the verifier reconstructs a faithful local subgraph $\mathcal{I}'$ of $\mathcal{I}$: for every node included in $\mathcal{I}'$, its authenticated structural information and its recovered distance to the query are identical to those in the authentic index.

Integrity of the Replay Process. Let f denote the designated deterministic ANN algorithm. In the Ideal Game, the trusted party outputs $R_{\mathsf{Ideal}} = f(Q, \mathcal{I})$. In the Real Game, once the structural proof and the aggregated IPA proof are accepted, the verifier replays the same deterministic algorithm on the reconstructed subgraph $\mathcal{I}'$ and obtains a local replay result $R' = f(Q, \mathcal{I}')$. The verification procedure accepts only if the replay does not require any missing node and $R' = R_{\mathsf{Real}}$.

We now analyze the ways in which Adv may try to make the Real Game output a result different from the Ideal Game.

Case 1: Adv provides an incomplete subgraph. Suppose Adv omits some node that is required by the honest execution of f on the authentic index. Because the ANN search proceeds by graph traversal, every such node is reached through the authenticated adjacency list of some previously revealed node. Hence, during replay on $\mathcal{I}'$, the verifier will eventually attempt to access a missing node. According to the verification procedure, this causes rejection, and thus $\mathbf{Game}_{\mathsf{Real}} = \perp$. Therefore, an incomplete subgraph cannot lead to a successful forgery.

Case 2: Adv provides a complete and faithful subgraph but deviates from the search protocol. Assume now that $\mathcal{I}'$ contains all nodes required by the honest search and is faithful to $\mathcal{I}$ on those nodes. Since the verifier replays the same deterministic ANN algorithm f on the same authenticated structure and distances, the replay necessarily follows the honest behavior on the revealed subgraph. Consequently, the replay result coincides with the honest result: $R' = R_{\mathsf{Ideal}}$.

If Adv outputs any forged result $R_{\mathsf{Real}} \neq R_{\mathsf{Ideal}}$, then necessarily $R_{\mathsf{Real}} \neq R'$, so the final equality check fails and the verifier rejects. Hence, in this case as well, Adv cannot make the Real Game output a value different from the Ideal Game while still being accepted.

Security of Topology-aware Instantiation. For the topology-aware instantiation, MeST replaces the standard MHT while preserving the binding between every authenticated graph-node opening and the signed root. Any modification to the revealed graph structure therefore implies either a hash collision or a signature forgery. Once the authenticated local subgraph is reconstructed, the replay argument is identical to that of the basic VIDA scheme.

Security of Quantization-aware Instantiation. For the quantization-aware instantiation, the replay process further contains two types of branches. For nodes represented by the exact-distance branch (*i.e.*, nodes in V_{exa}), the verifier follows the same exact-distance authentication procedure as in VIDA: the commitments, norms, and claimed inner products are authenticated by the ADS and the aggregated IPA proof, and hence the recovered exact distances are faithful to the committed vectors. Such authenticated exact distances are sufficient for replay even if a node could have been pruned by quantization. Specifically, if $\delta_j \leq \delta_{\mathrm{th}}$, then, under the stated guarantee of the quantization method, the pruning condition cannot hold for v_j, so the node cannot be validly pruned. Otherwise, if $\delta_j > \delta_{\mathrm{th}}$, the designated ANN update discards the node without changing the effective search state. Therefore, an unnecessary exact evaluation may change the verification branch, but it leaves the replay state and result unchanged.

For nodes pruned by quantization (*i.e.*, nodes in V_{qua}), the verifier checks the pruning decision using the authenticated quantization payload. Specifically, each node opening g_i^{op} is authenticated by the Merkle proof and binds, for every exposed neighbor v_j, both the exact-distance payload digest H_j^{exa} and the quantization payload digest H_j^{qua}. Therefore, the adversary cannot replace η_j without causing a hash collision or forging the signed root. The verifier recomputes $LB_j = \texttt{QuantLB}(\eta_j, \mathbf{x}_q)$ and accepts the pruning branch only if $LB_j > \delta_{\mathrm{th}}$ at the same replay point. Under the same quantization guarantee, this pruning decision is valid, and the pruned node cannot improve the current search state. Consequently, every accepted replay step either follows the designated branch or replaces a valid pruning step with a state-preserving exact evaluation. In either case, the verifier reaches

the same effective search state after processing the node and therefore obtains the same final result as the designated quantization-aware search.

Taken together, the aforementioned cases show that the only way for Adv to achieve $(\mathbf{Game}_{\mathsf{Real}} \neq \perp) \wedge (\mathbf{Game}_{\mathsf{Real}} \neq \mathbf{Game}_{\mathsf{Ideal}})$ with non-negligible probability is to break at least one of the following assumptions: the security of the signature, collision resistance of $\mathbb{H}$, computational binding of the commitment scheme, or computational knowledge soundness of the IPA protocol. All of these events occur only with negligible probability. Therefore, the success probability of Adv is negligible, which proves the theorem. □

APPENDIX D
PERFORMANCE RESULTS ON ADDITIONAL DATASETS

We report additional results on Wikipedia, Msmarco, and Cluster, complementing Figures 8 and 9. All experimental settings and methods follow Section VIII. In Figures 13 and 14, latency is decomposed into server-side verifiable search (Server), network transfer (Transfer), and client-side verification (Client).

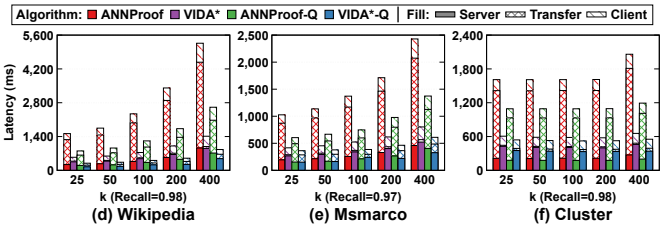


Fig. 13. Latency of Varying k: Additional Datasets

Impact of k. We vary k from 25 to 400 while maintaining target recalls for each dataset. As shown in Figure 13, VIDA* and VIDA*-Q achieve lower latency than their corresponding baselines across all tested k values. On Cluster, latency is nearly flat for $k = 25$ to 200 and increases at $k = 400$, consistent with the search effort needed to maintain a fixed target recall even at small k.

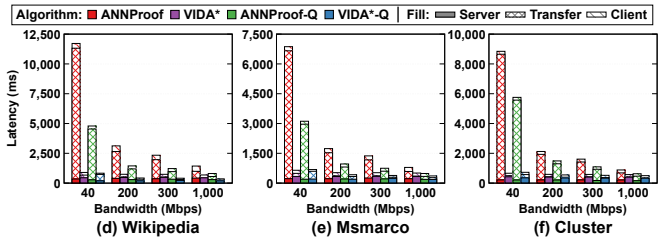


Fig. 14. Latency on Different Network Settings: Additional Datasets

Impact of Network Bandwidth. We fix $k = 100$ and use the same target recalls under four bandwidth profiles: 4G (40 Mbps), 5G (200 Mbps), Broadband (300 Mbps), and Server (1,000 Mbps). Figure 14 shows that VIDA* and VIDA*-Q outperform their corresponding baselines across all four bandwidth profiles, with the largest gains at 40 Mbps. Compact authentication payloads reduce the transfer cost that dominates baseline latency at this bandwidth.

APPENDIX E
ADDITIONAL EXPERIMENTAL RESULTS

A. Offline Construction Results

We evaluate the offline overhead of index initialization across the six datasets, measuring construction time and ADS size under the settings in Section VIII.

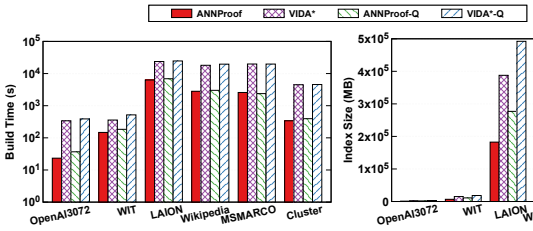


Fig. 15. ADS Build Time

Fig. 16. Index Size

Construction Time. As shown in Figure 15, ANNProof generally incurs the lowest construction overhead because it mainly relies on hashing. VIDA* additionally generates vector commitments and constructs the topology-aware ADS. Compared with their corresponding baselines, VIDA* and VIDA*-Q require 5.5× and 5.4× the construction time on average, respectively.

Storage Consumption. Figure 16 shows the storage cost of the additional authentication information. VIDA* stores vector commitments and the topology-aware ADS, while VIDA*-Q further stores quantization metadata for authenticated pruning. Their ADS sizes increase by 116.1% and 75.4%, respectively, over their corresponding baselines.

This offline cost is paid once by the data owner and is an acceptable trade-off for the substantial online latency and communication reductions reported in Section VIII.

B. Performance under Low Dimensions and Low Recalls

We further evaluate end-to-end query latency on low-dimensional Cluster and at lower recall targets across different values of k.

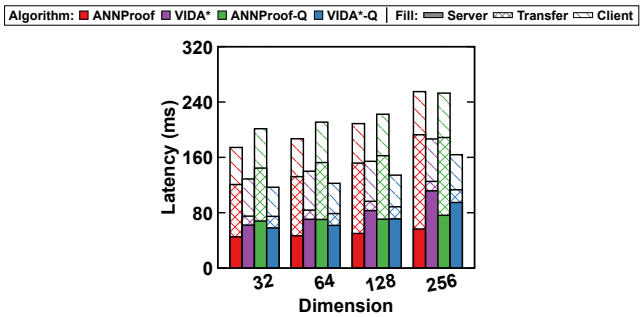


Fig. 17. Latency on Low-dimensional Cluster

Performance on Low-dimensional Cluster. Following the setup of the scalability study in Section VIII-C1, we extend the dimensionality experiment on Cluster to $32, 64, 128, 256$, keeping the search parameters unchanged. Figure 17 shows that VIDA* and VIDA*-Q achieve 1.34–1.37× and 1.54–1.72× speedups over their corresponding baselines, respectively, although the gains are smaller than at high dimen-

sions. Smaller raw-vector payloads reduce the communication savings from IPA-based authentication relative to its proving and verification costs. The remaining advantage is consistent with MeST's reduction of structural authentication overhead through selective neighborhood openings and shared Merkle paths. Compared with their non-quantized counterparts, ANNProof-Q is 15.5% slower and VIDA*-Q is only 9.3% faster at $d = 32$. Compressing already small vectors yields limited absolute savings, while authenticating quantization metadata and pruning decisions adds overhead. Thus, quantization helps only when its computational and communication savings outweigh the added authentication cost.

Fig. 18. Latency at Lower Recall Targets on OpenAI3072

Performance under Low Selectivity and Low Recall. We evaluate OpenAI3072 at $k = 10, 25, 50$ and target recalls of $0.75, 0.80, 0.85, 0.90$, keeping the bandwidth at 300 Mbps. Figure 18 shows that VIDA* remains faster than ANNProof across all settings, with speedups of 1.11–2.47×. However, VIDA*-Q is slower than ANNProof-Q at all four recall targets for $k = 10$ and 25, and at targets 0.75 and 0.80 for $k = 50$. For example, at $k = 10$ and target recall 0.75, ANNProof-Q takes only 118.0 ms, compared with 242.4 ms for VIDA*-Q. As k and the target recall increase, the latency gap between VIDA*-Q and ANNProof-Q narrows, with VIDA*-Q becoming slightly faster at $k = 50$ and target recalls 0.85 and 0.90. Under low selectivity or low recall, reduced search effort makes raw-vector transmission less costly, especially when quantization reduces the number of candidates requiring exact-distance verification. Under these lighter workloads, the communication savings from compact proofs may not offset proof generation and verification costs.